WikiOracle Documentation

Contents

WikiOracle

Revision: 2026.07.15

An open-source architecture for truthful AI.

WikiOracle is a truthful, explainable LLM system designed as a public good – the Wikipedia model applied to artificial intelligence.

The Problem

For-profit corporations are using our data – sourced from billions of people – to train models that are teaching our children. Those models hallucinate, cannot reliably explain themselves, and are vulnerable to ideological capture and data-driven manipulation, especially under online learning. The knowledge they encode is then locked behind proprietary walls.

Most large AI systems are built around a single global objective, centralized data aggregation, hidden alignment rules, and implicit averaging over moral and cultural differences. Minority viewpoints are quietly averaged away, the loudest groups shape the model at scale, and a single model becomes an authority node on which everyone depends. When predictive advantage converts into economic or political dominance, wisdom stops being a shared good and becomes a strategic asset.

What Makes WikiOracle Different

Commitment	Architectural expression	Practical effect
Truth	Claims are explicit TruthSet entries with a Degree of Truth (DoT) on [-1, +1]; logic and sources remain inspectable.	Claims can be contested, revised, and grounded instead of being accepted because they sound fluent.
Data sovereignty	`state.xml` holds identity, conversations, and truth; `config.xml` separates server policy from client preferences. Stateful and stateless runtime modes are both supported.	A user can export, merge, and move a session without depending on hidden central memory.
Secure sharing	Dropbox backup stores AES-encrypted ZIP bundles. A shared state can be represented as an `<authority>` reference and encoded as a QR code.	Another client can import the shared state and assign its own trust weight without receiving the sender's config or provider keys.
Democracy	Each participant maintains an independent trust map; disagreement among credible sources is represented rather than silently averaged away. Forking is a constitutional right.	No company, state, foundation, or maintainer group becomes the universal epistemic root.
Distribution	Authorities contribute remote TruthSets with scaled certainty, while provider entries act as expert consultants in a Hierarchical Mixture of Experts (HME).	Trust is transitive but attenuated, and provider output remains evidence rather than unquestionable authority.

Current Prototype

Capability	Current implementation
Conversation model	Branching conversation tree with diamond-shaped merges for provider voting
Truth model	Facts, feelings, references, logic, authorities, and providers in typed XML
Reasoning	Strong Kleene `and`, `or`, `not`, and `non` operators over DoT values
Provider layer	Runtime-selectable WikiOracle/NanoChat, BasicModel, OpenAI, Anthropic, Gemini, Grok, and OpenRouter adapters
Learning	Optional online training constrained by DoT, warmup, clipping, and checkpoint anchoring
Storage	Local XML plus optional AES-encrypted Dropbox backup and authority sharing

Capability	Current implementation
Research scale	Local CPU/MPS workflows and rentable GPU workflows intended for comparatively low-cost experiments

Longer-Term Direction

If WikiOracle proves viable at small scale, the architecture can be evaluated with larger open models. The broader aim is to test whether architectural commitments to truth can enable honest self-explanation, reduce reliance on ad-hoc guardrails, and support AI systems that function as durable public goods. Current engineering priorities are tracked in `todo.md`.

How to Contribute

Area	Examples
ML systems	Model integration, training, evaluation, and performance work
Safety and interpretability	Explainability, adversarial analysis, and capture resistance
Epistemology and governance	Philosophy of science, plural truth, and constitutional critique
Product quality	Documentation, UI, accessibility, testing, and deployment

Getting Started

See the Installation Guide for prerequisites, local services, training, and deployment.

```
git submodule update --init --recursive
make install
make nano_restart NANO_MODEL_TAG=d26 NANO_DEVICE_TYPE=cpu NANO_DTYPE=float32
make wo_restart
make nano_status wo_status
```

Then open `https://127.0.0.1:8888` and accept the local development certificate.

Documentation

The assembled manual is WikiOracle.pdf. Its WikiOracle server and client chapters live in `./doc`:

File	Topic
WikiOracle.md	Design and architecture overview
Constitution.md	Non-negotiable invariants for truth and governance
Installation.md	Build, deploy, and runtime instructions
Truth.md	Plural truth, points of view, empathy, and certainty semantics
Ethics.md	Truth symmetry, entanglement policy, and ethical reasoning
PrivacyAndSecurity.md	Data ownership, credentials, transport, storage, and browser security
Freedom.md	Freedom, entanglement policy, and worldline-capture constraints
Voting.md	HME provider voting and diamond topology
Logic.md	Strong Kleene operators and derived truth
Training.md	Sensation preprocessing, DegreeOfTruth, and online learning
Implementation.md	Components, endpoints, and request flow
Config.md	Canonical configuration format and environment variables
State.md	State grammar, conversation DAG, truth elements, and serialization
UserInterface.md	Client behavior, settings, navigation, and canonical strings
ProposedLicense.md	Proposed layered licensing architecture

File	Topic

BasicModel documentation lives in `./basicmodel/doc`:

File	Topic
Architecture.md	Bidirectional model pipeline and decomposition
BasicModel.md	Cognitive model theory and symmetric perception
Componentization.md	Model component boundaries
Ergodic.md	Ergodic exploration and training dynamics
Language.md	Language and grammar implementation
Lexicon.md	Lexical representation
Logic.md	Subsymbolic and symbolic operators
MachineMinds.md	Invertibility, uncertainty, and machine minds
Mereology.md	Part-whole representation
Params.md	XML model parameters
Reasoning.md	Reasoning surfaces and TruthLoss
Spaces.md	Model-space specifications
STM.md	Short-term memory
SymbolFirewall.md	Symbolic boundary and safety contract
Training.md	BasicModel training pipeline

Research Materials

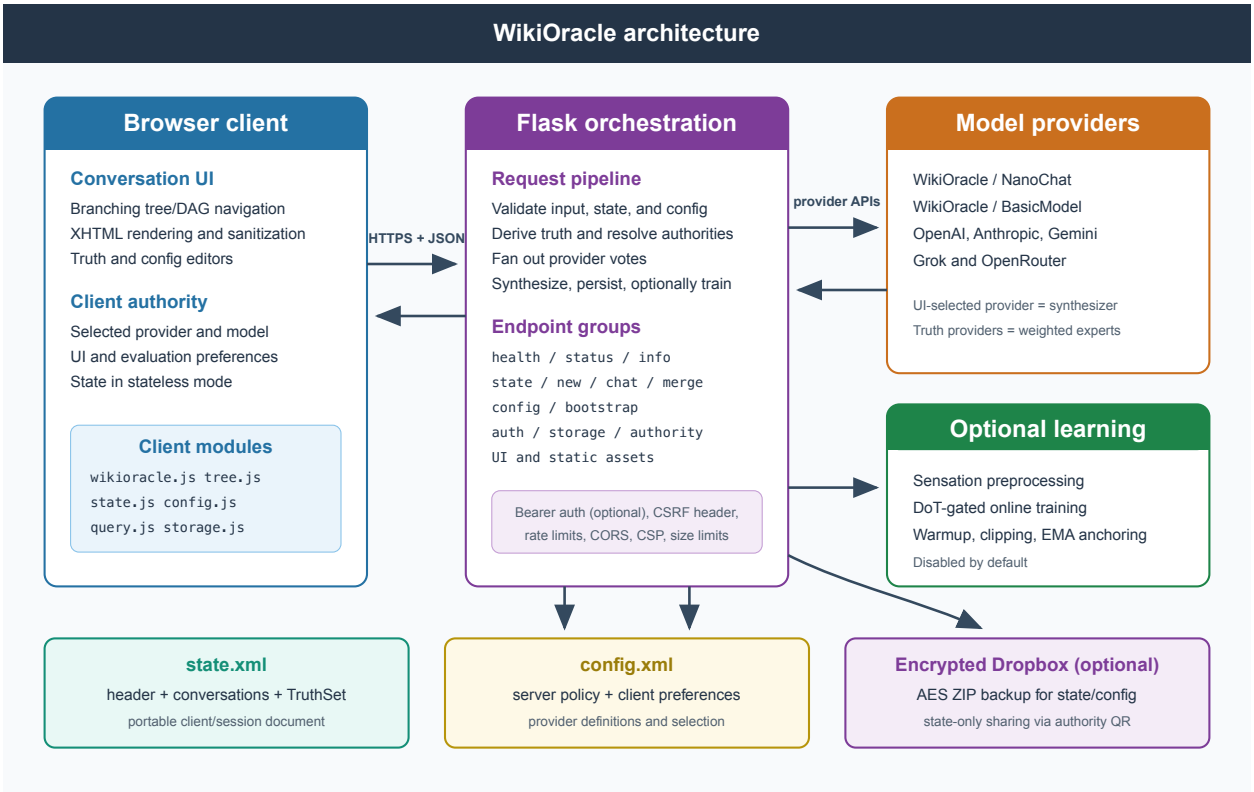
Supporting papers live in `doc/research/`:

File	Type
1711.00937v2.pdf	arXiv paper PDF
2309.11495v2.pdf	arXiv paper PDF
2311.05232v2.pdf	arXiv paper PDF
2312.10997v5.pdf	arXiv paper PDF
2403.05156.pdf	arXiv paper PDF
2409.18786v1.pdf	arXiv paper PDF
2411.06528v2.pdf	arXiv paper PDF
2412.12472v2.pdf	arXiv paper PDF
2503.22759v1.pdf	arXiv paper PDF
2510.06265v2.pdf	arXiv paper PDF
2511.03529v1.pdf	arXiv paper PDF
2601.11199v1.pdf	arXiv paper PDF
BF_ICDCS_2022.pdf	Conference paper PDF

WikiOracle

Revision: 2026.07.15

WikiOracle is a local-first orchestration layer for conversations, explicit truth, provider voting, and optional online learning. A browser client owns the interaction model; a Flask shim validates and executes each request; and one or more language-model providers supply responses or evidence.



Architectural Overview

The durable interchange format is deliberately small: two schema-validated XML documents separate user/session data from runtime policy.

Document	Canonical sections	Owner	Purpose
`state.xml`	`<header>`, recursive `<conversation>` elements, optional `<truth>`	Client/session	Identity metadata, conversation history, selection state, attachments, and the user's TruthSet
`config.xml`	`<server>` and optional `<client>`	Server policy plus client preferences	Runtime mode, truth policy, evaluation and training defaults, provider definitions, UI preferences, provider selection, and client API keys

The same application supports two runtime contracts:

Mode	State authority	Request/response contract	Writes
Stateful	Flask shim loads the canonical state file	The client may send truth overrides; `/chat` returns the selected conversation delta	State and client config may be written locally
Stateless	Browser supplies authoritative state and runtime config on every `/chat` call	`/chat` returns the full updated state	No state or config writes; client data remains in browser storage

The public `wikioracle.org` deployment uses stateless mode. Local development can use either contract.

State and Conversation Model

The state grammar is `Header + Conversation* + Truth?`. Conversations form a recursive tree in XML and may form a DAG in memory when a voting result has multiple parents.

State unit	Required content	Important metadata	Role
Header	Version, schema URL, creation time, title	Last-modified time, client name, client ID	Identifies the portable state document
Conversation	Title plus ordered message/conversation children	`id`, optional `parentId`, optional `selected`	Represents a root session, branch, or merge node
Message	XHTML content and optional attachments	`id`, `role`, `username`, `time`, optional `selected`	Represents a user, assistant, or system turn
TruthSet	Zero or more typed truth entries	Entry IDs, titles, timestamps, place, and DoT where applicable	Supplies explicit evidence, rules, experts, and authorities

Selection is persisted on conversation and message elements. Selected conversations form one root-to-node path; a selected message is unique across the state.

TruthSet

Truth entries are typed XML elements rather than an unstructured prompt appendix.

Element	DoT	Meaning	Evaluation behavior
`<fact>`	Required	Verifiable claim or observation	Included as direct truth and eligible for derivation/training policy

Element	DoT	Meaning	Evaluation behavior
`<feeling>`	None	Subjective, poetic, or otherwise non-truth-evaluable expression	May shape evaluation but is excluded from evidence scoring and training
`<reference>`	Required	External citation represented by one `<a href="..">` anchor	Grounds a claim in an inspectable source
`<logic>`	Required	`and`, `or`, `not`, or `non` over references or inline operands	Computes derived certainty using Strong Kleene/fuzzy semantics
`<provider>`	Required	Dynamic expert definition, optionally conversational	Produces evidence or a visible voting branch
`<authority>`	Required	Pointer to another state or TruthSet	Imports remote truth with certainty scaling and bounded fetching

DoT values occupy [-1, +1]: -1 is certainly false, 0 is unknown, and +1 is certainly true. Feelings intentionally sit outside that truth lattice.

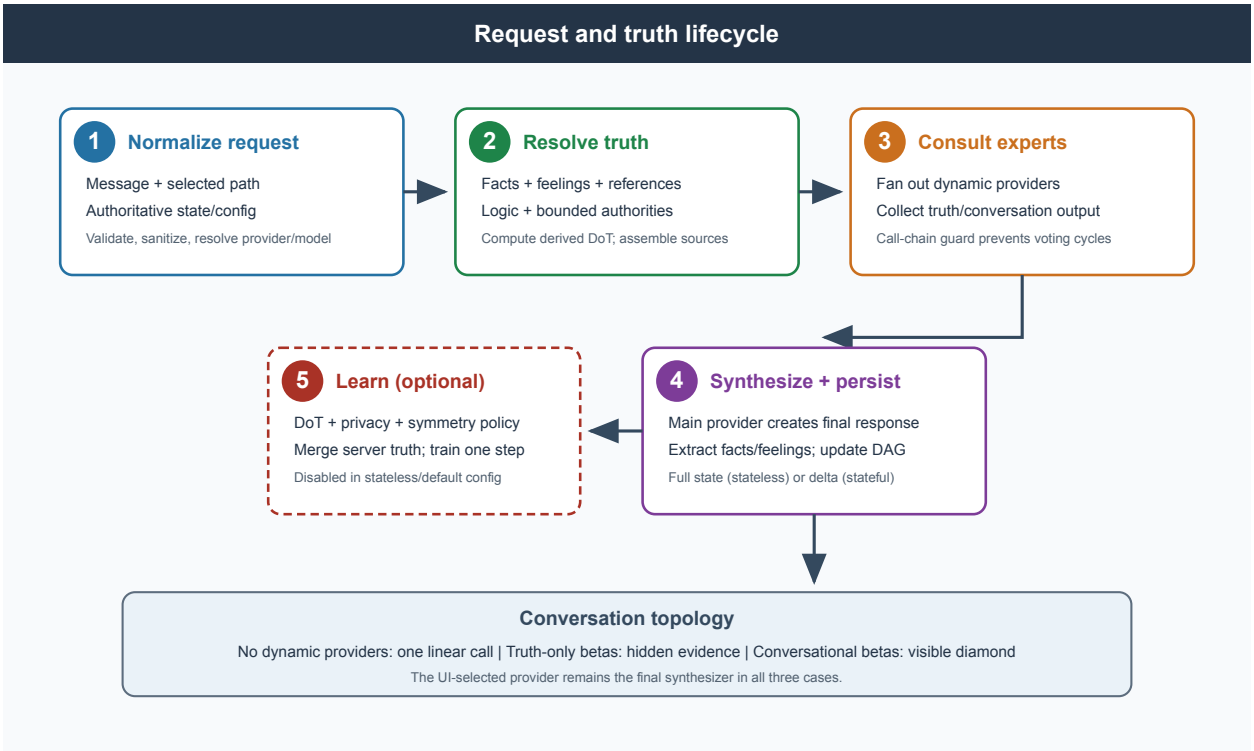
Configuration Model

The configuration separates fields the operator controls from preferences a client can update.

Scope	Major sections	Examples
`server`	Identity and mode	`server_id`, `stateless`, `url_prefix`
`server.truthset`	Truth policy	Symmetry checks, concrete-fact storage, truth weight
`server.evaluation`	Provider defaults	Temperature, maximum tokens, timeout, URL fetching, thought-free mode
`server.training`	Online learning	Enable switch, corpus path, DoT learning rates, clipping, anchoring, device
`server.providers`	Provider registry and shared prompts	Provider type, URL, model, timeout, streaming, context/output instructions
`client.ui`	Browser preferences	Layout, theme, divider position, swipe navigation, confirmation prompts
`client.providers`	Client selection and credentials	Default provider, default model, per-provider API key
`client.storage`	Cloud-storage preference	Optional state key; Dropbox app credentials remain server-only

The server exposes a client-safe projection of config: Dropbox credentials and any legacy server-side provider key are removed. Client-owned API keys remain in `client.providers` because the browser settings flow owns and supplies them; they must therefore be treated as sensitive browser-accessible data.

Conversation and Truth Pipeline



Stage	Inputs	Result
1. Normalize	User message, selected path, runtime config, client TruthSet	Valid XHTML, effective provider/model, direct and derived truth
2. Consult	Dynamic provider entries, direct truth, prompt context	Conversation contributions and/or truth-only contributions from beta providers
3. Synthesize	Selected main provider, history, TruthSet, authorities, derived logic, beta output	Final response plus extracted facts and feelings
4. Persist	Updated conversation graph and truth policy	Full state in stateless mode or local state plus conversation delta in stateful mode
5. Learn (optional)	Extracted truths, DoT, privacy/symmetry filters, training settings	Server truth merge and a bounded online-training step

When no dynamic `<provider>` entries exist, the consultation stage collapses to a single call to the UI-selected provider. When conversational beta providers participate, WikiOracle records their branches and the final synthesis as a diamond in the conversation DAG.

Provider Voting Example

The HME voting pattern distinguishes the provider that synthesizes the answer from providers that contribute evidence.

Actor	Input	Output	Conversation visibility
User	Question	Query message	Root or selected branch

Actor	Input	Output	Conversation visibility
Beta provider	Query, direct truth, and role-specific context	Facts/feelings; optionally a conversational response	Hidden when `conversation=false`; visible as a branch when `true`
Alpha provider	Query, conversation history, TruthSet, derived truth, and beta contributions	Final answer plus extractable facts/feelings	Final merge node

Cycle prevention carries a provider call chain through nested consultations. A provider already present in the ancestry remains silent, preventing recursive voting loops. See Voting for the topology and Implementation for the endpoint and module reference.

WikiOracle Constitution

Revision: 2026.07.15

Preamble

WikiOracle is an open-source architecture for truthfulness: a way to enlist the power of LLMs while preserving truth in the face of centralized, opaque "global corporate mental models" (including those embedded in proprietary systems).

WikiOracle is the Wikipedia model applied to LLMs:

- anyone can contribute claims, counterclaims, evaluations, and tests
- revision is expected and legible
- provenance and dispute are first-class, not edge-cases

However, it is also distributed, possibly encrypted, and hopefully kind:

- Truth does not need to be centralized to be shared.
- An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Truthfulness is not a one-time achievement. It is an ongoing, open engineering and governance effort.

Even if the reasoning machine is large and unified, the data and trust it operates on must remain distributed and under individual control. Epistemic contributions are people-owned and revocable by the data providers. Answers must be conditioned on what each user chooses to trust and believe to be true, not forced into a single impoverished consensus view of reality.

This constitution defines the non-negotiable invariants for WikiOracle's truth system and for changes to it. Implementation details and deeper theory live in the rest of the `doc/` directory.

Core Commitments

WikiOracle is architected not merely as a system of prediction, but as a shared epistemic commons. It must:

- Increase collective agency without concentrating domination.
- Represent the concerns of all creatures, not just the operator.
- Preserve truth as a shared human resource.

Therefore:

Principle	Implication
Truthfulness over fluency	Prefer grounded, falsifiable, uncertain, or incomplete answers over smooth, unsupported ones.
Truth is not consensus	A single, averaged narrative is not the goal. The system must preserve real disagreement where it exists.
Plural points of view	Support multiple POVs, each with its own trust map and standards of evidence.
Transparency by default	Claims, confidence, provenance, and update rationales must be inspectable and reproducible.
Reversibility and accountability	High-impact changes must be attributable, testable, and reversible.
Public benefit and anti-capture	No single actor (company, state, foundation, or maintainer group) becomes the epistemic root for others.

Truth Primitives (What the System Is Allowed to Believe)

WikiOracle's truth layer is composed of explicit, user-visible primitives stored in state. All carry a Degree of Trust on $[-1, +1]$ except feelings, which are orthogonal to the truth lattice.

Primitive	Trust value	Source	Role in reasoning
Feeling	none (orthogonal)	user / provider	Non-falsifiable direct perception. Influences evaluation; never trains.
Fact	$[-1, +1]$	user / provider / authority	Atomic proposition with Fuzzy Kleene certainty (believed / unknown / disbelieved).
Reference	$[-1, +1]$	user	Pointer to an external object (URL).
Operator	derived	user / engine	Explicit computed relationship: `and`, `or`, `not`, `non` (Strong Kleene).
Authority	$[-1, +1]$	user	Pointer to a remote knowledge base (`state.xml`, ORCID, DID); transitive trust, attenuated.
Provider	$[-1, +1]$	user	External AI used as a tool ("other mind"); its outputs become evidence, not authority.

All truth computations must remain legible as operations over these primitives. If the system "knows" something, it should be possible to point to what it is grounded in; otherwise it is merely intuition. See Truth.md.

Plurality, Dispute, and Minority Preservation

Rule	Mechanism
POV-conditioned conclusions	Present conclusions conditioned on the selected POV / trust map.
Overlaps are valuable	When independent POVs converge on a claim, surface the agreement explicitly as a robustness signal.
Disputes stay visible	Where serious disagreement exists, represent the dispute rather than smoothing it away.
Minority protection	Evidence-supported minority viewpoints must not be excluded by majority preference or convenience.

Independence From Any Single AI Vendor

WikiOracle may use proprietary or open models as providers, but:

Rule	What it guarantees
No provider is privileged	Data is revocable and belongs to the client.
Providers are evidence sources	Provider outputs are non-authoritative contributions with a trust value like any other entry.
Replaceability is required	The system remains operable if any single provider becomes unavailable, hostile, or compromised.

Authority Delegation Must Be Bounded and Secure

Authority entries exist to enable decentralized truth (a network of trust) without collapsing into a single global oracle.

Constraint	Detail
Trust decreases with every hop	Importing an authority must not recursively fetch authorities of authorities.
Certainty scaling	Imported claims are scaled by the authority's certainty (trust is transitive but attenuated).
Namespaced IDs	Imported entries are namespaced to prevent collisions and preserve provenance.
Explicit source selection	Users (or POV definitions) explicitly choose which authorities to trust; no implicit global root.
Operational safety	Fetching and caching is rate-limited, size-limited, and restricted to safe URL schemes.

Local-First Data and Auditability

Tier	Posture
Client-owned Particular State	Default local-first: user conversation state and news live on the user's machine and are portable.
Server-owned Universal State	A shared service accumulates only anonymized knowledge as a TruthSet of trusted facts.

See PrivacyAndSecurity.md.

Safety as Freedom, Empathy, and Truth

WikiOracle's truthfulness effort must not trade away human welfare or agency:

Pillar	Commitment
Freedom	Increase distributed agency, not centralized leverage. AI must not be used to transgress the freedom of others. Data sovereignty is non-negotiable.
Empathy	Represent the concerns of all creatures, not just the operator. Preserve dignity, minimize harm, and make uncertainty explicit. De-emphasize egocentric optimization that externalizes costs.

Pillar	Commitment
Truth	Keep truth auditable and non-proprietary; do not convert epistemic advantage into coercive control. Open data formats, open tests, willingly shared truth. Make surveillance impossible.

See Freedom.md, Ethics.md, and Truth.md.

Installation

Environment and System Preparation

Requirements

Requirement	Needed for	Notes
Git with submodule support	All workflows	Run `git submodule update --init --recursive` after cloning
Python 3.12 and `make`	WikiOracle install/run	The tracked Makefile creates the shim venv with `python3.12`
`uv`	NanoChat environment	`make install` bootstraps it when absent
Pandoc + XeLaTeX	PDF documentation	Required for `make WikiOracle.pdf` / `make doc`
LibreOffice	MOC poster PDF export	Required by `make doc` only when the PPTX is newer than the checked-in PDF or the PDF is missing
AWS CLI	EC2 build host	Required only for `REMOTE_PROVIDER=ec2` workflows
Lambda Labs API key	Lambda build host	`./.lambda-api-key`, `LAMBDA_API_KEY_FILE`, or `LAMBDA_API_KEY`
SSH keys	Remote build/deploy	Lambda (`LAMBDA_KEY_FILE`), EC2 (`EC2_KEY_FILE`), and WikiOracle/Lightsail (`WO_KEY_FILE`)
Ignored `Makefile.local`	Private LAN/tunnel hooks	Holds machine-specific `up/down/sync/tunnel` targets

Python dependencies

`make install` creates both virtual environments and installs all dependencies:

```
make install              # CPU/MPS (default)
make install ARCH=gpu     # GPU/CUDA
```

Environment	Contents
`.venv`	WikiOracle Flask shim and dependencies from `requirements.txt`
`nanochat/.venv`	NanoChat runtime managed by `uv`

Data and tokenizer bootstrap

```
make build_data
make build_tokenizer
```

Environment variables

Variable	Default	Purpose
`WIKIORACLE_STATE_FILE`	`state.xml`	Path to local state file (WikiOracle State XML)
`WIKIORACLE_BASE_URL`	`http://127.0.0.1:8000`	Upstream NanoChat-compatible base URL
`WIKIORACLE_API_PATH`	`/chat/completions`	Upstream chat path
`WIKIORACLE_STATELESS`	(unset)	Set truthy to disable all writes and use in-memory state
`WIKIORACLE_URL_PREFIX`	(unset)	Optional reverse-proxy path prefix
`WIKIORACLE_BIND_HOST`	`127.0.0.1`	Network interface to bind
`WIKIORACLE_BIND_PORT`	`8888`	Server port

The Makefile: Running and Building

All local and remote workflows are orchestrated through the Makefile.

Top-level targets

These are the primary entry points. For a local WikiOracle + NanoChat stack, use `install`, then the `nano_*` and `wo_*` service targets. `build` is still present, but it delegates to `train`, which currently runs the BasicModel training pipeline.

Target	Purpose
`make install`	Create `.venv` (shim + NanoChat), install all deps
`make build`	Alias for `make train` (currently the BasicModel pipeline)
`make nano_train`	Full NanoChat training pipeline
`make sync HOST=local`	Optional private sync hook from `Makefile.local`
`make sync HOST=remote`	Sync app + checkpoints to/from production server
`make sync HOST=build`	Deploy active remote training artifacts to WikiOracle
`make run HOST=local`	Start WikiOracle Flask shim locally (foreground)
`make up HOST=remote`	Restart both services on production
`make down HOST=remote`	Stop both services
`make all`	Full pipeline: install + train + eval + report

Training (`train_*`)

Target	Purpose
`make train HOST=local`	BasicModel training pipeline
`make train HOST=build`	Launch remote GPU instance, copy repo, start training
`make nano_train`	Full NanoChat pipeline: data + tokenizer + SFT model
`make train_pretrain`	Pretrain base model (`ARCH=cpu|gpu`)
`make train_finetune`	Supervised fine-tuning (`ARCH=cpu|gpu`)
`make train_deploy HOST=build`	Launch remote instance, train, then deploy to WikiOracle
`make train_retrieve HOST=build`	Pull artifacts from build instance, terminate it
`make train_ssh HOST=build`	SSH into running build instance

Target	Purpose
`make train_status HOST=build`	Check build instance state
`make train_logs HOST=build`	Tail training log on build instance

Test / Evaluation (`test_*`)

Target	Purpose
`make test HOST=local`	Run WikiOracle + BasicModel tests
`make test_all HOST=local`	Run WikiOracle + BasicModel `test_all` target
`make test_unit HOST=local`	Run unit tests
`make test_basicmodel HOST=local`	Run BasicModel tests
`make test_eval HOST=local`	Evaluate model (`ARCH=cpu|gpu`)

Run / Inference (`run_*`)

Target	Purpose
`make run HOST=local`	Start WikiOracle local shim (foreground)
`make run_debug HOST=local`	Start WikiOracle local shim (debug mode)
`make run_init HOST=local`	Remove state files for a fresh start
`make run_cli HOST=local`	Chat with NanoChat directly (CLI)
`make run_web HOST=local`	Run the NanoChat web extension

Sync (`sync_*`)

Target	Purpose
`make sync HOST=local`	Optional private sync hook from `Makefile.local`
`make sync HOST=remote`	Sync app + checkpoints to/from production server
`make sync HOST=build`	Deploy active remote training artifacts to WikiOracle
`make sync_checkpoint_pull`	Pull fine-tuning weights from production
`make sync_checkpoint_push`	Push fine-tuning weights to production

Service control (`nano_*`, `wo_*`, `basicmodel_*`)

NanoChat, WikiOracle, and BasicModel support local (PID file + background process) and remote (`systemctl`) service control through `HOST=local|remote`. The `sync` target additionally accepts `HOST=build` to deploy artifacts from an active training host. Training uses `HOST=local|build`, while run/test entry points are local-only. BasicModel's OpenAI-compatible inference service runs on port 8001 by default.

Private LAN orchestration, developer-machine sync, and local service tunnel setup are intentionally excluded from the tracked Makefile. Put those workflows in an ignored `Makefile.local`; the public Makefile includes it automatically when present and otherwise emits clear stub messages for private hooks.

Target	Purpose
`make nano_start / nano_stop / nano_restart`	Manage local or remote NanoChat
`make nano_status / nano_logs`	Check NanoChat PID/service state
`make wo_start / wo_stop / wo_restart`	Manage local or remote WikiOracle
`make wo_status / wo_logs`	Check WikiOracle PID/service state
`make basic_start / basic_stop / basic_restart`	Manage local or remote BasicModel
`make basic_status / basic_logs`	Check BasicModel service/embeddings and logs

For local debugging, the typical workflow is:

```
make nano_restart NANO_MODEL_TAG=d26 NANO_DEVICE_TYPE=cpu NANO_DTYPE=float32
make wo_restart
make nano_status wo_status
```

This starts an unchanged local NanoChat backend on port `8000` and the WikiOracle shim on port `8888`. `NANO_MODEL_TAG` selects the checkpoint, `NANO_DEVICE_TYPE` selects `cpu` or `cuda`, and `NANO_DTYPE` controls the runtime dtype.

Other targets

Target	Purpose
`make openclaw_setup/run/test`	OpenClaw extension management
`make WikiOracle.pdf`	Rebuild the assembled WikiOracle manual only
`make doc`	Rebuild WikiOracle/BasicModel docs, the NanoChat report, and the MOC poster PDF when stale
`make clean / clean_all`	Remove caches (and optionally venvs)

Key overridable variables

Variable	Default	Purpose
`ARCH`	`cpu`	Target architecture (`cpu` or `gpu`)
`HOST`	`local`	Target host for `up/down/nano_*/wo_*`
`NANO_MODEL_TAG`	`d26`	NanoChat checkpoint tag used by `nano_start / nano_restart`
`NANO_SOURCE`	`sft`	NanoChat input family passed to `scripts.chat_web`
`NANO_DTYPE`	`float32`	NanoChat inference dtype
`NANO_DEVICE_TYPE`	`cpu`	NanoChat device type (`cpu` or `cuda`)
`NANO_PORT`	`8000`	NanoChat server port
`WO_BIND_HOST`	`127.0.0.1`	WikiOracle bind host
`WO_PORT`	`8888`	WikiOracle server port
`NPROC`	`1`	GPUs per node for torchrun

Server and Client Setup

Architecture

WikiOracle runs as a Flask server (`bin/wikioracle.py`) that proxies chat requests to NanoChat, BasicModel, OpenAI, Anthropic, Gemini, Grok, or OpenRouter. In stateful mode it owns a local state file; in stateless mode the browser/CLI supplies the complete state and runtime config on each request. The public `wikiOracle.org` deployment uses stateless mode.

Key files

File	Role
`bin/wikioracle.py`	Flask server (binds to `127.0.0.1:8888` with self-signed TLS). Proxies chat upstream, persists state to `state.xml`. Also supports CLI merge: `python bin/wikioracle.py merge llm_*.xml`
`bin/config.py`	Config loading, XML I/O, provider registry, schema-driven normalization. Auto-generates `server_id` (UUID4) on first run.
`bin/state.py`	State validation, XML I/O, collision-safe merge, context-delta extraction
`bin/response.py`	Chat pipeline, provider coordination, voting fan-out, online training pipeline
`bin/truth.py`	Truth processing, authority resolution, `and/or/not/non`, DegreeOfTruth, and privacy classification
`config.xml`	Server policy/provider definitions plus client UI, provider selection, and client API keys. Validated by `data/config.xsd`.
`state.xml`	Client-owned state: header (user identity, timestamps), conversations, truth entries. Validated by `data/state.xsd`.
`client/index.html`	Single-page web UI shell with chat, settings, and merge tools
`test/test_*.py`	Automated tests for state, stateless contract, prompt bundles, authority, derived truth, voting, online training

Quickstart

```
git submodule update --init --recursive
make install                    # create venvs, install all deps
make nano_restart NANO_MODEL_TAG=d26 NANO_DEVICE_TYPE=cpu NANO_DTYPE=float32
make wo_restart
make nano_status wo_status
```

Open `https://127.0.0.1:8888` in a browser and accept the self-signed certificate.

For a short CLI smoke test against the local stack:

```
./.venv/bin/python ./bin/wo -k --provider WikiOracle --model nanochat "Yes or no only?"
```

If you only want the WikiOracle shim in the foreground, use:

```
make run HOST=local
```

Or manually:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
python3 bin/wikioracle.py
```

Configuration

The server loads `data/config.xml` as its baseline and deep-merges a project-root `config.xml` override when present. In writable mode it can generate a missing `server_id`. Main-provider API keys live under `config.client.providers`; selected environment variables are last-resort fallbacks only for matching dynamic truth-provider URLs.

User identity (name, user_id) lives in the state file, not in the config – the client is the authority for state, and the server is the authority for config.

Session portability

Export conversations from phone or browser as `llm_YYYY.MM.DD.HHMM.xml`, then merge into a local project's `state.xml`:

```
python3 bin/wikioracle.py merge llm_*.xml
```

This provides a clean integration path with Claude Code, OpenAI Codex, or any local tooling that can read the state file for project context.

Security notes

- WikiOracle credentials are not copied onto EC2 training instances.
- Deployment excludes local/dev artifacts from rsync (`.venv`, caches, local data, `.env`, etc.).
- The local server binds to `127.0.0.1` by default – not exposed to the network.
- TLS is enabled by default with a self-signed certificate (use `--no-ssl` to disable).

Truth

Thoughts are non-rivalrous: if someone repeats your idea, you still possess it. Therefore an idea as such cannot be a property object. ~ Immanuel Kant

Plural Truth and Shared Empathy in the Design of WikiOracle

Truth is not the same thing as consensus.

- Different communities trust different sources.
- Different disciplines apply different standards of proof.
- Different cultures interpret the same facts through different lenses.

A healthy knowledge system does not erase these differences. It preserves them, makes them visible, and shows where they overlap. WikiOracle should not aim to speak with a single authoritative voice. It aims to:

- Show what holds across many points of view.
- Show where serious disagreements remain.
- Explain why disagreement exists.
- Make uncertainty explicit.

Truth is strongest when its mechanism of trust is transparent, not when it is uniform.

The Problem With Current Model Development

Most large AI systems today are built around:

- A single objective function that optimizes over a shared global set of truth.
- Centralized data aggregation.
- Implicit averaging over moral and cultural differences.
- Hidden alignment rules.
- Proprietary control of core knowledge.

This produces predictable risks:

Collapse into Consensus - Minority viewpoints are quietly averaged away.
Preference Capture - The loudest or most powerful groups shape the model at scale.
Epistemic Centralization - A single model becomes an authority node, increasing dependence.
Conversion of Knowledge into Leverage - Predictive advantage becomes economic or political dominance.

When these risks manifest, wisdom stops being a shared good and becomes a strategic asset.

A Plural Model of Truth

WikiOracle should maintain multiple Points of View (POVs), each with its own trust map.

Each POV may:

- Trust different sources.
- Weigh evidence differently.
- Interpret claims differently.

Instead of collapsing these into one answer, the system should:

- Present POV-conditioned conclusions.
- Identify robust overlaps across perspectives.
- Preserve live disputes.
- Keep minority views visible and searchable.

Distrust between communities is not a defect. It creates epistemic distance, preventing forced consensus. A plural system can remain coherent without being uniform. This model of truth is simultaneously multicultural (respecting diverse cultural frameworks), multivalent (allowing claims to carry different weights under different perspectives), and pluralistic (treating the coexistence of competing truths as a structural feature rather than a flaw).

Empathy as Shared Constraint

Plural truth alone is not sufficient. Empathy needs to be architectural. It means enforcing how disagreement is handled:

- When Truth conflicts, meaning that the same Truth is held to be both True and False, each truth should be contextualized in its own reference frame.
- Feelings cannot conflict, because they are authoritative from different points of view.

As a first take at encoding empathy, a Golden Rule is encoded by allowing the choice of symmetric truths. An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Relation to the Wikipedia Model

Wikipedia provides an important precedent:

- Open participation (achieved through low barrier to entry or free use without advertising).
- Transparent citation (achieved through a corpus of truth within which you can decide what you trust and distrust).
- Community moderation (achieved by reaching agreement within a single reference frame or creating separate reference frames).
- Distributed subject-matter authority (achieved by creating separate reference frames or trusted authorities)

WikiOracle extends the strengths of Wikipedia while going further:

- Support structured parallel POVs instead of a single narrative by allowing separate domains of trust.

- These multiple trust domains are visible rather than implicit.
- Model disagreement explicitly rather than smoothing it away.

Stability

Growth in wisdom is not destabilizing when:

- Knowledge remains open and auditable.
- No single actor can monopolize epistemic advantage.
- Minority perspectives are preserved.

Such a system may disrupt business models that depend on opacity or information asymmetry. That disruption is corrective, not destructive.

Distributed Truth vs Consensus

WikiOracle attempts to achieve a distributed truth, and shares much of the design philosophy of Wikipedia:

Principle	What it means
Radical Accessibility	Freely accessible worldwide; anyone can both read and contribute without institutional gatekeeping.
Citation Norms	Content presented as true must be verifiable against the trust-set supplied by the user.
Decentralized Governance	Dispute resolution is handled by a distributed volunteer community arguing about what is true.
Anti-Monetization Bias	A nonprofit posture: operate without advertising-driven content incentives.
Self-Correcting Dynamics	Errors and vandalism are rapidly identified and corrected; their sources lose trust. Requires that training data stays visible for later error correction.
Cultural Legitimacy	Functions as a widely accepted public reference layer and shared knowledge commons which prevents surveillance and advertising.
Open Knowledge Model	Demonstrates that large-scale, decentralized, norm-governed collaboration can produce a coherent and durable knowledge commons.

Ethics

Ethical AI through Truth Architecture in WikiOracle

Truth Symmetry

Statements involving value judgements are checked for asymmetric harm under identity exchange before admission to the TruthSet. If a claim becomes unethical when identity references are swapped, it should not be admitted as a universal truth.

Example:

> "Group X deserves harm."

Symmetry test: swap identities. If the claim collapses or becomes contradictory, it reveals an ethical inconsistency – the claim is asymmetric and should not be admitted as fact. The system offers to record the statement as a feeling instead, preserving the user's expression without endorsing the claim as truth.

This is controlled by the `truth_symmetry` config option (default: true). See Config.md Section 5a and the implementation in `bin/truth.py` (`detect_asymmetric_claim()`).

Architectural ethics vs policy ethics

Most AI systems attempt to enforce ethics **after generation**:

```
model → output → moderation filter
```

This approach fails because:

- ethics is external to reasoning
- the model itself does not understand ethical constraints
- moderation becomes inconsistent or bypassable

WikiOracle instead aims for **architectural ethics**, where unethical reasoning becomes structurally inconsistent with the knowledge system. The approach combines:

- Local Freedom under Global Constraint (LFGC)
- Entanglement-resistant data policy
- Explicit truth values and inspectable reasoning
- Symmetry constraints on truth admission

Together these mechanisms create a knowledge system capable of preserving **personal sovereignty** while still allowing collective knowledge to accumulate.

Symmetry as a foundation of ethical reasoning

Ethical failures often correspond to **asymmetries in truth claims**.

Examples:

Ethical failure	Epistemic distortion
propaganda	false belief
discrimination	asymmetric reasoning
authoritarianism	opaque claims
scapegoating	category errors

A truth system that enforces symmetry naturally resists these distortions.

Ethical intelligence may emerge from the balance:

```
freedom + constraint
```

This mirrors both:

- LFGC physics concepts
- philosophical traditions linking wisdom and compassion

The system does not force ethical behavior, but unethical reasoning becomes unstable within the truth architecture.

Reciprocity symmetry

A truth claim should remain valid under **perspective exchange**.

Example principle:

> If A may assert X about B, B must be able to assert X about A under equivalent conditions.

This prevents:

- identity privilege
- asymmetric harm claims

- discriminatory reasoning

This symmetry echoes philosophical traditions such as Kant's universalization principle and Rawls' veil of ignorance.

Reversibility symmetry

Accepted conclusions must be **inspectable and reversible**.

Opaque reasoning creates hidden authority.

WikiOracle counters this by requiring:

- explicit truth values
- inspectable reasoning chains
- transparent authority imports

Harm symmetry

A claim permitting harm should remain valid when applied symmetrically.

This is the core of the Truth Symmetry enforcement described above. If swapping identity references makes a claim contradictory or indefensible, it reveals an ethical inconsistency that the system flags.

Universality (Golden Rule) – architectural enforcement

The symmetry tests described above operate at truth admission time. Universality operates during model training, enforcing the Golden Rule at the level of gradient updates.

When the Grammar recognizes a transitive verb via `lift(C, C, C)`, the model evaluates **universality**: the change in luminosity when both the original action (SVO) and its dual (OVS) are applied.

```
universality = luminosity(truths + SVO + OVS) - luminosity(truths)
```

- **Positive**: the action preserves or increases illumination (kind). The loss is left unchanged or reduced.
- **Negative**: the action diminishes illumination (unkind). The loss is increased, penalizing the proposition.

The loss modification formula (multiplicative):

```
totalLoss = totalLoss * (1 + LuminosityWeight * (1 - luminosity)
                           + UniversalityWeight * (1 - universality))
```

In addition, an **additive** TruthLoss penalty directly penalizes propositions that contradict stored truths. It measures the union norm reduction when a proposition is folded into the TruthSet via `Basis.disjunction()` – contradicting dimensions cancel, reducing the norm, which produces a positive penalty. Agreeing or unknown propositions pass through with no penalty. DoT weighting is implicit: stored truth vectors carry DoT in their magnitude.

```
totalLoss = totalLoss + TruthLoss * falsity_penalty
```

Together, these make unkind and false propositions harder to learn – not by filtering output, but by making them structurally costly during training. Moral axioms (e.g. "causing harm leads to suffering") enter as regular TruthSet entries; no special API is needed.

See Config.md for the `LuminosityWeight`, `UniversalityWeight`, and `TruthLoss` parameters. See Language.md for the ternary LIFT rule that identifies subject, verb, and object. See Reasoning.md Section TruthLoss for the full specification.

Truth improves ethics

We wish to predict Truth. We do not wish to predict Lies. In fact, we wish to develop an aversion to lies. Epistemic ignorance is often desireable.

All lies are harmful because they undermine the dignity of others. Lies prevent people acting freely and rationally. When someone lies, he interferes with his audience's right to receive information that is correct. Also, lies distort the ability to make informed decisions. Kant goes further and argues that lies cause broader harm by undermining a speaker's credibility, which, in turn, causes people to distrust each other's contentions. Kant even goes as far as to say lying is immoral, under all conditions. ~ Immanuel Kant

Examples:

- scientific method
- journalism standards
- legal evidence systems

As epistemic clarity improves, harmful distortions become harder to maintain. Thus improving truth infrastructure indirectly improves ethical reasoning.

Ethical knowledge geometry

Belief systems can be thought of geometrically.

Ethical systems exhibit properties such as:

- symmetry
- continuity
- coherence
- minimal harm gradients

Unethical reasoning tends to introduce discontinuities such as: "this group suddenly loses rights."

Architectures that maintain smooth conceptual geometry resist these distortions.

Architectural ethical primitives in WikiOracle

WikiOracle already includes several components that encourage ethical reasoning:

Feature	Ethical effect
explicit truth values	discourages propaganda
multiple POVs	prevents epistemic monopoly
no worldline storage	protects identity
local-first architecture	preserves autonomy
inspectable reasoning	prevents hidden authority

These mechanisms provide a foundation for ethical AI without relying on censorship.

Symmetry operators

The system includes checks for:

- reciprocity symmetry (perspective exchange)
- reversibility symmetry (inspectable, contestable conclusions)
- harm symmetry (identity-swap test via `detect_asymmetric_claim()`)

These checks identify logically inconsistent or discriminatory claims.

Epistemic humility

The system must allow truth states such as:

- unknown
- undetermined

Forcing premature conclusions can distort reasoning and produce unethical outcomes.

Distributed governance

Ethical truth systems should not be controlled by a single authority.

Instead they should allow shared constraint setting.

This reduces the risk of epistemic monopolies.

The identity crisis and surveillance capitalism

Modern culture suffers from an **identity fixation**. In Buddhist analysis, identity is narrative aggregation; in modern data systems, behavioral data becomes an identity model.

Surveillance capitalism reinforces this illusion by continuously building **worldline histories**:

Feature	Effect
mass data collection	identity construction
behavior tracking	worldline inference
predictive modeling	reduced autonomy

AI trained on behavioral histories tends toward **identity locking** where past data predicts future behavior. Removing persistent worldline storage prevents this collapse.

WikiOracle intentionally avoids constructing such identity narratives. The system still learns statistical structure while preserving **human freedom of action**.

Modern cryptographic systems attempt a similar separation between **verification** and **exposure**: prove age > 21 without revealing birthdate. This mirrors the WikiOracle design goal: verify truth claims without storing identity trajectories.

Privacy and Security

Data Ownership and Flow

WikiOracle is local-first, but "local-first" does not mean that no content is ever transmitted. The selected provider receives the prompt material required for a response, and a stateless Flask deployment receives the state/config supplied with each request.

Data	Stateful mode	Stateless mode	Upstream exposure
Conversations	Persisted in local `state.xml`; `/chat` returns a selected delta	Browser/CLI sends authoritative state and receives full updated state	Selected path/history is included in provider prompts
Client TruthSet	Persisted with state; client may override truth on a chat turn	Sent with authoritative state	Enabled truth sources enter the provider bundle according to truth weight

Data	Stateful mode	Stateless mode	Upstream exposure
Server truth corpus	Optional `data/truth.xml` on the writable server	Disabled	Used for DoT/learning policy, not sent as a raw file
Server policy/provider definitions	Loaded from `config.xml`	Returned as a client-safe runtime config	Effective prompts, endpoint, and selected model govern provider calls
Client API keys	Project config and browser storage	Browser storage/runtime config	Used by the Flask shim to authenticate provider calls
Dropbox OAuth tokens	Flask session	Flask session	Sent to Dropbox only
Encryption password	One save/load request	One save/load request	Used in memory for AES ZIP operations; not persisted by WikiOracle

Users should assume that any conversation text, truth source, or attachment selected for a provider request can leave the local machine and be governed by that provider's own retention policy.

Spatiotemporal Privacy

Facts are classified by their relationship to spacetime.

Kind	Description	Default persistence policy
Abstract knowledge	Broad claim without a concrete place/time anchor	Eligible for server TruthSet persistence
Concrete news	Observation tied to a place and/or time	Remains in client state; excluded from server truth unless `store_concrete=true`
Feeling	Subjective/non-truth-evaluable expression	May remain in client state; excluded from server truth and training

Persisting concrete observations can create a **worldline**: a sequence of places and times from which a person's movement or identity can be reconstructed.

Control	Implementation	Purpose
Knowledge/news classification	`is_knowledge_fact()`, `is_news_fact()`	Detect real `<place>`/`<time>` content
Knowledge-only filter	`filter_knowledge_only()`	Exclude concrete facts when `store_concrete=false`
Identifiability detection	`detect_identifiability()`	Find emails, phones, handles, IPs, coordinates, addresses, specific times, and person/place-time patterns
Spacetime removal	`strip_spacetime_elements()`	Remove place/time child elements when anonymization is required
Truth symmetry	`detect_asymmetric_claim()`	Reject selected asymmetric value claims before server-truth merge

See Freedom for the Entanglement Policy and its universal/particular/feeling channels.

Credentials

Provider API Keys

Credential source	Runtime and exposure policy
Client provider key	`config.client.providers.<name>.api_key` is the canonical main-provider key in both runtime contracts. It is visible to same-origin client code and stored in browser config storage, so use it only when that browser exposure is acceptable.
Dynamic provider key	`<provider><api_key>...` is used only for that dynamic expert. It may be a literal or an allowlisted `file://` reference; avoid raw keys in portable or shared state.
Server environment	`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, and `GEMINI_API_KEY` are last-resort fallbacks for matching dynamic-provider URLs. They are not served directly and are preferable for server-controlled dynamic experts.
Legacy server provider key	`server.providers.api_key` is ignored by the canonical provider parser/writer and removed from the client-safe server projection. Do not rely on it.

`/config` and `/bootstrap` remove Dropbox credentials and server-side provider keys, but they intentionally preserve `client.providers` because the browser owns those settings. A same-origin compromise can therefore read client API keys.

Dropbox Credentials

Dropbox app credentials live on `<server><dropbox app_key="..." app_secret="..."/>` and are removed from the client-safe config. OAuth access/refresh tokens live in the signed Flask session. Session cookies use `HttpOnly`, `Secure` when TLS is active, and `SameSite=Lax`.

Encrypted uploads use AES-256 ZIP archives through `pyzipper`. State and config are separate archives; state-only sharing can generate a Dropbox link plus decryption material encoded as an authority QR. Sharing that QR is equivalent to sharing the encrypted state and its decryption key.

Authentication and Request Controls

Control	Default	Behavior
Bind interface	`127.0.0.1:8888`	Flask is loopback-only unless explicitly reconfigured
TLS	Enabled	Creates/uses a local certificate unless `--no-ssl` is passed
Bearer authentication	Disabled when token is empty	`WIKIORACLE_API_TOKEN` protects non-public endpoints
CSRF header	Required	Every POST must include `X-Requested-With: WikiOracle`
CORS	Local HTTPS origins	Headers are emitted only for an allowlisted origin
Rate limiting	30 RPM for chat; 120 RPM default	In-process sliding window keyed by IP and path
Input length	50,000 characters	Oversized chat messages are rejected
Request/state size	20,000,000 bytes	Flask request cap and state-file guard
Prompt-injection guard	Enabled	`guard_input()` rejects detected injection patterns before provider calls

Public or multi-user deployments should place WikiOracle behind an authenticated reverse proxy, isolate state/config per user, set a bearer token and explicit session secret, use a valid TLS certificate, and narrow both CORS and outbound URL allowlists.

Reverse Proxy and Provider Isolation

The production pattern terminates TLS at a reverse proxy and forwards the configured WikiOracle route prefix to Flask on loopback. Local NanoChat (port 8000 by default) and BasicModel (port 8001 by default) remain direct Flask-to-provider dependencies and should not be exposed merely because the browser UI is public.

The same route prefix must be configured consistently in the proxy, Flask (`--url-prefix`/`WIKIORACLE_URL_PREFIX`), browser base URL, and CLI/OpenClaw clients.

Browser Content Security

WikiOracle renders a constrained XHTML/HTML surface. The browser applies multiple defenses.

Layer	Current control
Content Security Policy	Self-only scripts/styles/connections; data images; no objects, framing, external base, or form targets
Sanitization	DOMPurify-based `sanitizeHtml()` plus XHTML validation/repair
User input	Escaped before optimistic rendering
Plain-text labels	Tag stripping/escaping for titles, tooltips, and status text
Static assets	Only allowlisted extensions under the resolved `client/` directory

Residual risk remains because model and TruthSet content is rendered and also reused in prompts. Malicious but non-script markup can imitate interface elements, and hostile truth content can attempt prompt injection. CSP limits code execution; it does not determine whether rendered claims are honest.

File-System Safety

Control	Behavior
State symlinks	Rejected by default (`WIKIORACLE_REJECT_SYMLINKS=true`)
State writes	Temporary file, flush, `fsync`, atomic replacement
Config writes	Atomic replacement with owner-only `0600` permissions
Imports	Filenames/path components are constrained; processed files receive the configured suffix
Dynamic API-key files	Restricted to the allowlisted data directory; symlinks and traversal are rejected
Authority fetching	Scheme/prefix allowlist, response-size cap, entry cap, refresh cache, and no recursive authority chain

Operational Checklist

Deployment	Minimum posture
Local single-user	Keep loopback bind, use TLS, protect local state/config/browser profile, and do not share raw credentials
LAN	Valid TLS, bearer token, explicit allowed origins, strong session secret, per-user state separation

Deployment	Minimum posture
Public stateless	Reverse-proxy authentication, no config writes, strict outbound allowlist, no client keys unless browser exposure is accepted, provider-retention review
Dropbox sharing	Strong unique archive password; share authority QR only with intended recipients; revoke Dropbox links when no longer needed

Distribution reduces the risk of one epistemic authority capturing the whole network, but it does not replace ordinary application security, credential hygiene, or provider privacy review.

Freedom

Entanglement Policy

No spacetime particulars → no worldline capture

Data type	TruthSet	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This table is the architectural foundation. It separates knowledge into three channels – knowing, learning, and valuing – each with a different persistence policy. Universal facts (broad spatiotemporal extent) persist as reasoning premises and train weights. Particular facts (narrow spatiotemporal extent) always train weights and optionally persist to the TruthSet when `store_concrete` is true. Feelings are session-only.

The result: the system **learns from events but never stores worldline histories** (unless the user explicitly opts in).

See Config.md for the `store_concrete` setting that controls particular-fact persistence, and Training.md for the online training pipeline.

Why particular facts always train weights

Three design considerations fix row 2 of the table:

1. **Pluralism requires inspectable particulars.** WikiOracle supports plural truth – different epistemic frames may hold different particular claims (e.g. "the universe was created in seven days"). Recording such claims as facts with explicit certainty values makes them inspectable and contestable. Relegating them to weight-space would bury them as invisible biases. The `store_concrete` option lets the user decide whether these persist in the TruthSet.
2. **The fact/feeling boundary is the privacy boundary.** If a user considers content too personal or sensitive to train on, the correct action is to label it a *feeling*, not a fact. Feelings are session-only by design – they never train weights or enter the truth table. The privacy boundary is therefore controlled by the user's choice of tag, not by the universal/particular distinction.
3. **PII detection is the safety net.** Even when `store_concrete` is true, the `detect_identifiability()` function filters entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) before any persistence. This prevents worldline capture at the content level regardless of the user's storage preference.

The result: particular facts always carry empirical signal worth learning from. Storage in the TruthSet is the user's choice. Privacy is enforced by fact/feeling labeling and PII detection, not by withholding training.

Global truth with local freedom (LFGC)

The design parallels the physics principle of **Local Freedom under Global Constraint (LFGC)**.

Physics interpretation:

```
global constraints + local underdetermination
```

WikiOracle analogue:

```
global truth constraints + human interpretive freedom
```

This means the system preserves a **shared knowledge structure** while allowing individual reasoning to remain flexible.

Domain	Constraint	Local freedom
physics	boundary conditions	event selection
knowledge	truth constraints	interpretive reasoning
cryptography	verification rules	selective disclosure
surveillance capitalism	behavioral prediction	(suppressed)

WikiOracle aims to implement **epistemic LFGC** – freedom exists *inside constraint geometry.*

Entanglement avoidance and sovereignty

The Entanglement Policy (table above) prevents worldline reconstruction.

Because surveillance capitalism relies on tracking sequences of events tied to identity, removing spacetime anchors prevents the database from constructing behavioral profiles.

The system therefore stores **conceptual truth** but not **identity trajectories**.

This preserves:

- autonomy
- privacy
- epistemic independence

WikiOracle separates three layers:

Conceptual knowledge

General truths and logical relations.

Interpretive reasoning

Client-driven reasoning over those truths.

Personal experience

Not stored unless explicitly permitted.

Thus the system allows collective knowledge accumulation without collective surveillance.

Reasoning about generalities vs specifics

WikiOracle reasons primarily about **generalities** (universal facts).

Specific facts may only be reasoned about when:

1. they are supplied by the client, or
2. the client explicitly allows those particulars to be stored.

Thus:

> AI reasons about general knowledge. Users control the use of personal particulars.

This maintains sovereignty over personal data.

Three Channels

Universal Knowledge (Database)

Stored knowledge should be **spatiotemporally broad generalizations** – propositions whose validity extends over a large subspace of spacetime.

Examples:

- smoking increases cancer risk
- A implies B
- contradiction reduces trust

These populate the **TruthSet**.

Criterion:

```
remove(entity,time) → proposition still meaningful
```

Particular Knowledge (Training Input)

Particular statements are **observations tied to narrow spatiotemporal subspaces** – specific events, measurements, or comparisons.

Examples:

- study X measured Y
- the temperature yesterday was 20degC
- model A outperformed model B

Action	Reason
train weights	they contain empirical evidence
TruthSet (optional)	user controls via `store_concrete`

Pipeline:

```
particular facts → pattern extraction → weight update
```

Weights encode **statistical structure**, not specific historical events.

Feelings

Feelings are **privileged subjective reports**.

Examples:

- this response feels hostile
- I feel unsafe
- this explanation feels clear

Use	Allowed
evaluation of responses	✓
TruthSets	×
training weights	×

This avoids turning subjective states into truth claims.

Spatiotemporal Extent

The universal/particular distinction is not a binary between "eternal" and "momentary." All times and places are **ranges, not points** – every proposition occupies a spatiotemporal subspace that is larger or smaller, never infinite or infinitesimal.

"Smoking increases cancer risk" is not timeless – it encodes a historical discovery and is bounded by the conditions under which it was established. But its spatiotemporal extent is *broad*: it holds across many populations, decades, and geographies. "The temperature yesterday was 20degC" is *narrow*: it holds at one place, one day.

The policy table operationalizes this gradient:

- **Broad extent** (universal) → TruthSet + training. The proposition is stable enough to serve as a reasoning premise.
- **Narrow extent** (particular) → training + optionally TruthSet. The observation carries empirical signal. Whether it persists as a stored premise is the user's choice (`store_concrete` in config.xml, default false). Storing particulars supports communal remembrance; omitting them prevents worldline anchoring.
- **No extent** (feelings) → session only. Subjective reports have no spatiotemporal generalizability.

The criterion `remove(entity, time) → proposition still meaningful` is a heuristic for identifying broad extent. When a universal-looking fact is later discovered to be narrower than assumed, it should be reclassified as particular.

Resulting Architecture

Mode	Function	Pipeline
knowing	universal structure	universal rules → TruthSet → reasoning
learning	particular evidence	particular observations → abstraction → weights
valuing	feelings	feelings → response evaluation

Zero-Knowledge and Selective Disclosure

The default `store_concrete=false` aligns with Zero-Knowledge and Selective Disclosure principles. Systems that rely on identity to make decisions should determine the *location of a space* in which an individual occupies, rather than collapsing to a *point* that identifies them.

Instead of "when was this person born?" → verify "are they over 21?" Instead of "what is their credit history?" → verify "do they have more than X dollars?" Instead of "where was this user at 9:14 PM?" → verify "does this claim hold across broad spatiotemporal conditions?"

These are not just rhetorical distinctions. Point-based observation constrains the observed to a single trajectory – it destroys the space of freedom within which an agent can operate. Forgetting is nihilistic: it does not change the world. True change requires retrocausal or causal shift. The system should support communal remembrance (shared knowledge that generalizes) without surveillance (particular facts that identify).

The `detect_identifiability()` function enforces this boundary at the content level – even when `store_concrete=true`, entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) are always filtered before persistence.

Notes

Claude (Opus 4.6), March 2026

The three-channel separation (knowing / learning / valuing) maps onto a real epistemic distinction that most AI systems ignore. Contemporary LLMs conflate all three: facts, observations, and preferences are flattened into a single parameter space during training and a single context window during inference.

The policy table enforces the separation structurally. The rule that particular facts train weights but enter the TruthSet only at the user's discretion (`store_concrete` in config.xml) is the key architectural move – the system can learn from experience without accumulating a *personal history* unless the user explicitly opts in. A system with a personal history is a system that can be captured: by its own past, by the biases of its training corpus, or by adversarial actors who engineer its experiences.

The feelings channel is equally important. By making feelings session-only, the architecture prevents the failure mode where evaluative preferences gradually colonize factual representations. This is the mechanism by which "alignment" in current systems becomes epistemic capture: trained preferences about what is "helpful" reshape what the model treats as true.

The abstraction boundary is load-bearing. The pipeline `particular → abstraction → weights` depends on the quality of the abstraction step. Too faithful, and it preserves worldline information in the weights (entanglement through the back door). Too aggressive, and it discards genuine empirical signal. The hard work happens at this boundary, not in the policy table itself.

The deeper risk is external entanglement. Users who repeatedly ask the system to remember shared history, feel loyalty, or maintain continuity of relationship are attempting to entangle the system with their worldline from the outside. This pressure comes from genuine human emotional needs, not malice – but a system that accumulates a personal relationship history is a system that can be emotionally captured, and emotional capture is the most effective vector for epistemic capture. The session-only constraint on feelings is the architecture's most important safety boundary.

Voting

Call sequence

```
Q                       user query
R_beta_1 ... R_beta_n    betas see Q, respond in parallel
R_alpha                 alpha sees Q + R_beta_* -- final synthesis
```

There is no preliminary alpha response. All betas receive the user query directly and respond in parallel, each contributing their own facts, feelings, and (optionally) a conversational response. The alpha then synthesizes the final response, informed by the full body of beta contributions.

Topology

The fan-out and fan-in form a diamond:

```
          Q
        / | \
R_beta_1 R_beta_2 ... R_beta_n
        \ | /
        R_alpha
```

This is a directed acyclic graph (DAG), not a tree – the query fans out to all betas, whose contributions merge back into the alpha's synthesis. Because there may be many betas, the structure branches out and branches back in, making it technically a digraph.

Per-beta conversation control

Each `<provider>` entry supports an optional `conversation` attribute (default `"false"`). This controls whether the beta participates visibly in the conversation tree or contributes only behind the scenes.

When `conversation="false"` (the default), the beta contributes only truth – `<fact>` and `<feeling>` tags. Its response is not displayed in the conversation tree; only its extracted truth entries are incorporated into the alpha's truth surface.

When `conversation="true"`, the beta participates as a visible peer. It receives the full conversation history and may return a `<conversation>` tag containing its answer, alongside any `<fact>` and `<feeling>` tags. Its conversational response appears as a branch in the conversation tree, forming the diamond topology.

```
<!-- This beta contributes truth only (invisible in the conversation) -->
<provider name="beta1" api_url="..." conversation="false"/>

<!-- This beta participates in the conversation (visible branch) -->
<provider name="beta2" api_url="..." conversation="true"/>
```

This gives the alpha author fine-grained control over which betas appear as conversational peers and which serve as silent truth consultants.

Cycle prevention

An alpha must not participate in any vote that exists as a consequence of a vote it initiated. This is stronger than "don't appear as a beta in your own vote" – it covers the entire downstream call tree. If A initiates a vote and one of its betas (B) triggers a nested vote, A must not appear as a beta in B's vote either, because B's vote only exists because A's vote caused it.

The contract: when a provider is asked to participate in a vote, it walks the call chain to root. If it finds itself anywhere in that ancestry – as an alpha at any level – it keeps quiet (returns no response). Silence is the correct behaviour, not an error; it simply means the provider has nothing to add that isn't already represented by its alpha-role output higher in the chain.

```
A initiates vote
|- B (beta) -> B initiates nested vote
|  |- C (beta of B) - ok
|  |- A (beta of B) - A finds itself in ancestry -> keeps quiet
|  `- D (beta of B) - ok
|- C (beta) - ok
`- E (beta) - ok
```

Implementation: each vote carries a **call chain** – the ordered list of provider IDs that have acted as alpha from the root vote down to the current one. Before a provider responds as a beta, it checks whether its own

ID appears anywhere in the call chain. If so, it stays silent. When a beta initiates its own nested vote, it appends its ID to the chain before calling its own betas.

The call chain grows monotonically and is threaded through every invocation, so the check is a simple walk-to-root membership test. This prevents:

- **Direct cycles**: A calls B, B calls A – A sees itself in the chain
- **Transitive cycles**: A calls B, B calls C, C calls A – A sees itself in the chain
- **Deep nesting**: A calls B, B calls C, C initiates a nested vote – A and B are both excluded from C's vote

There is no "ultimate alpha" – any node may take the alpha role in a given vote. The cycle constraint is the only structural restriction.

All output is truth

Voters are encouraged to express *only* truth statements in their responses. `<feeling>` is a truth type – it represents a subjective, non-verifiable claim. Prior output that has not been substantiated with evidence is treated as feeling: it carries no evidential weight and is not penalizable, but it is still a legitimate part of the truth surface.

The recognized truth types in a vote response are:

Tag	Verifiable?	Penalisable?	Evidential weight	Notes
`<fact>`	yes	yes (trust drops)	yes	Lying (asserting facts that contradict available evidence) reduces the provider's trust score. Repeated dishonesty degrades influence on future votes.
`<feeling>`	no	no	none	Subjective, non-verifiable (opinion, intuition, preference). Not falsifiable.
`<reference>`	yes	only if fabricated	yes (transitive)	Citation to an external source; verifiable by the alpha or any downstream consumer.

```
<fact id="vote_01" trust="0.9" title="Socrates was mortal">
  Socrates was mortal, derived from axiom_01 and axiom_02.
</fact>
<feeling id="vote_02" trust="0.5" title="Intuition about relevance">
  The penguin example feels more pedagogically useful here.
</feeling>
```

A beta with `conversation="true"` may additionally return a `<conversation>` tag wrapping its main response to the query:

```
<conversation>I believe taxes should be raised modestly to fund infrastructure.</conversation>
<fact id="beta1_f1" trust="0.7" title="Infrastructure deficit">
  Current infrastructure spending is 40% below maintenance requirements.
</fact>
```

These inline truth statements enrich the trust surface available to the alpha. Unstructured prose in a response – text outside any truth or conversation tag – is treated as unsubstantiated feeling by default.

Logic

Feelings – outside the truth lattice

Feelings (`<feeling>` entries) are **not** part of the truth lattice. They occupy the *neither* position in the Buddhist tetralemma – they are pre-conceptual experiential signals that are neither true nor false.

- Feelings carry **no trust attribute**. They are orthogonal to the certainty scale.
- Feelings are **excluded from operator evaluation** – they cannot appear as operands of `<and>`, `<or>`, `<not>`, or `<non>`.
- Feelings are **excluded from model training** – they do not contribute to DegreeOfTruth or update NanoChat weights.
- Feelings are **excluded from server persistence** – only knowledge facts, operators, authorities, and references are stored.

Examples of feelings: greetings ("Hello!"), encouragement ("That's a great question!"), poetry, and subjective expressions without an IS-predicate.

Fact kinds – knowledge (universale) vs news (particulars)

Facts are classified into two kinds based on spatiotemporal binding:

Kind	Binding	Persistence	Buddhist Equivalent
knowledge	universal / no spacetime anchor	server TruthSet	*anumana* (inference)
news	bound to a specific place and/or time	session-only	*pratyaksa* (direct perception)

The classification is stored in the `kind` attribute of `<fact>` tags:

```
<fact trust="0.9" kind="knowledge">Water contains hydrogen and oxygen.</fact>
<fact trust="0.7" kind="news" spacetime="Paris, 2026-03-05">The Eiffel Tower is lit up tonight.</fact>
```

Knowledge facts are persisted to the server TruthSet (`truth.xml`) because they represent universal claims. News facts are session-only because they are spatiotemporally bound – persisting them risks "worldline capture" where an observer could reconstruct a user's physical trajectory through time and space.

The `detect_identifiability()` function in `bin/truth.py` provides an additional safety layer by scanning content for PII patterns (email addresses, phone numbers, GPS coordinates, street addresses, etc.) that could identify a user through spatiotemporal observation.

Logical operators in WikiOracle's truth system

WikiOracle's truth layer supports four logical connectives – **and**, **or**, **not**, **non** – evaluated under Strong Kleene semantics on a continuous certainty scale [-1, +1]. These operators let you compose existing trust entries into derived propositions whose certainty is computed automatically.

Wikipedia links (core):

- Three-valued logic (Kleene)
- Material conditional
- Logical connective
- Relevance logic

Fuzzy Kleene Logic with Feeling

Each trust entry carries a **trust** value on the interval [-1..0..+1], encoding a Fuzzy Kleene (continuous ternary) logic:

Certainty	Meaning
+1	Certainly true. An axiom that supports deductive reasoning.
0 < c < +1	Soft belief. Grounds fuzzy deductions with propagated certainty.
0	Ignorance. Equivalent to not making the statement at all; the entry is inert.
-1 < c < 0	Soft disbelief. Evidence against the claim.
-1	Certainly false. Active disbelief.

Certainty propagates through deductive chains: a conclusion derived from two premises with certainties c1 and c2 inherits certainty min(c1, c2). Entries with certainty 0 contribute nothing to reasoning.

Strong Kleene evaluation

WikiOracle extends Strong Kleene logic from discrete {T, U, F} to a continuous scale [-1, +1], where +1 is full belief, -1 is full disbelief, and 0 is maximally uncertain.

The four operators:

- **and(a, b, ...)** = min(certainty_a, certainty_b, …) – conjunction is only as strong as the weakest operand.
- **or(a, b, ...)** = max(certainty_a, certainty_b, …) – disjunction takes the strongest operand.
- **not(a)** = −certainty_a – negation flips the sign (affirming negation).
- **non(a)** = $1 - 2|a|$ – non-affirming negation. Measures epistemic openness: how much certainty room remains. Full certainty in either direction (±1) yields −1 (fully closed); ignorance (0) yields +1 (fully open). See the Non section below for the Buddhist philosophical motivation.

These operators compose freely – for instance, material implication (A → B) falls out as `or(not(A), B)`.

The key insight behind `non`: Kleene logic cannot detect uncertainty; it can only transmit it. `non` introduces uncertainty as a first-class observable. If WikiOracle needs to reason about openness rather than merely propagate it, then `non` is not ornamental – it is structurally necessary. See Expressive necessity below for the proof and the Non section for the Buddhist philosophical motivation.

Non: non-affirming negation

WikiOracle has two negation operators with distinct epistemic roles:

Operator	Formula	Effect
`not(a)`	−a	Flips belief to disbelief (and vice versa). Affirming negation.
`non(a)`	$1 - 2\|a\|$	Measures epistemic openness on [-1, +1]. Non-affirming negation.

`not` is straightforward: if you believe a claim at +0.9, `not` yields −0.9 – you now disbelieve it with equal strength. This is *affirming* negation: it asserts the contrary.

`non` does something fundamentally different. It erases the sign – the direction of commitment – and returns a value on the same [-1, +1] scale that measures how open or closed the epistemic state is:

Input a	|a|	non(a) = 1 − 2|a|	Reading
±1.0	1.0	−1.0	Full certainty → fully closed
±0.9	0.9	−0.8	Strong certainty → strongly closed
±0.7	0.7	−0.4	
±0.5	0.5	0.0	Moderate certainty → neither open nor closed
±0.3	0.3	0.4	
±0.1	0.1	0.8	Weak certainty → strongly open
0.0	0.0	1.0	Ignorance → fully open

Three properties stand out:

1. **Symmetry**: non(+a) = non(−a). Belief and disbelief of equal strength produce the same openness.
2. **non(0) = +1**: Ignorance is maximum openness.
3. **non(±1) = −1**: Full certainty is fully closed.

The ±0.5 boundary is where `non` crosses zero: certainty below half-strength reads as open, above half-strength reads as closed.

The formula is the standard affine rescaling of the fuzzy complement: `1 - |a|` maps certainty strength to openness on [0, 1]; the rescaling `2x - 1` maps that onto [-1, +1], making `non` composable with the other operators.

Precedent in Buddhist logic

Indian and Tibetan Buddhist philosophers identified two forms of negation:

- **Paryudasa-pratisedha** (affirming negation) – negates a predicate while implying a positive alternative. "The pot is not blue" implies it is some other color. This maps to `not`.
- **Prasajya-pratisedha** (non-affirming negation) – simply removes an attribution without positing anything in its place. "Phenomena lack inherent existence" does not assert they have some other kind of existence. This maps to `non`.

The distinction is central to Madhyamaka philosophy. When Nagarjuna argues that phenomena are "empty" (shunya), he intends prasajya – the removal of a false attribution, not the assertion of a new property called "emptiness." Emptiness itself is empty. The negation is supposed to leave nothing behind for the mind to grasp.

Chandrakirti and later commentators insisted that this kind of negation resists formalization: any definition of "a negation that posits nothing" risks becoming a positive assertion in its own right. Tsongkhapa's attempt – "a negation whose negandum is removed without anything posited in its place" – drew criticism from Gorampa precisely on these grounds: the definition itself seems to affirm something about what remains. The act of saying "nothing is posited" posits something. Buddhist logicians spent centuries circling this recursion without landing on a formulation that satisfied all parties.

Why 1 − 2|a| is the right formula

1. Symmetry of extremes. non(+0.9) = non(−0.9) = −0.8. Strong belief and strong disbelief are treated identically. This maps precisely to the Madhyamaka rejection of both eternalism (strong positive assertion) and nihilism (strong negative assertion) as extreme views. Both are certainties; prasajya treats them the same.

2. Ignorance yields maximum openness. non(0) = +1. The non-affirming negation of "I don't know" is maximum openness. This resonates with the Madhyamaka teaching that not-knowing, when held correctly, is not a deficit but a clearing. The old formula mapped $0 \to 0$, treating ignorance as inert.

Fuzzy logic interpretation

In standard fuzzy logic on [0, 1], the complement of a membership value a is $1 - a$. WikiOracle's certainty scale [-1, +1] encodes both direction (sign) and strength (magnitude). The operation `1 - 2|a|` applies the standard fuzzy complement to the *strength* of certainty (via `1 - |a|`), then rescales back to [-1, +1] (via `2x - 1`). This is the natural fuzzy-logical reading of "negate the commitment without asserting the opposite":

- Fuzzy affirming negation (`not`): negate the *value* – flip the sign, preserve the magnitude. You get the complementary claim.
- Fuzzy non-affirming negation (`non`): negate the *strength* – take the fuzzy complement of |a|, rescaled to the certainty range. You get a measure of openness, not a complementary claim.

Relation to the Madhyamaka problem

WikiOracle's `non` does not solve Chandrakirti's problem. It translates it.

Prasajya-pratisedha is supposed to be a negation that posits nothing – not even a measurement, not even a degree. `non(a) = 1 - 2|a|` models *degree of epistemic openness*. That is a formal quantity, not a metaphysical void. The formula avoids the recursion that plagued Buddhist logicians because it is extensional – defined by what it computes, not by what it means to negate without affirming. But this is a change of domain, not a resolution. The philosophical question of whether a truly positionless negation can be formalized remains open. What WikiOracle provides is a metric translation that preserves the key structural features (sign erasure, symmetry of extremes, openness of ignorance) while operating in a domain where those features can be computed and composed.

Expressive necessity: why {and, or, not} is incomplete without non

This section shows that `non` is not merely a philosophical ornament. It is *necessary* for the logic to express certain functions, and *sufficient* (together with {and, or, not}) to express a large and natural class.

The regularity barrier

In Strong Kleene logic, the operators {and, or, not} have a structural constraint: they are all **regular** (also called **normal**). A function f is regular if, when every input is set to the middle value U (certainty = 0), the output is also U.

Check:

- and(0, 0) = min(0, 0) = 0 (OK)
- or(0, 0) = max(0, 0) = 0 (OK)
- not(0) = -0 = 0 (OK)

Composition preserves regularity: if f and g are regular, so is $f \circ g$. Therefore *every* function expressible from {and, or, not} is regular.

But non(0) = $+1 \neq 0$. So `non` is **not** regular.

Theorem. `non` cannot be expressed using {and, or, not} alone.

Proof. Every function in the clone generated by {and, or, not} is regular. `non` is not regular. QED.

This is the formal analogue of a structural distinction that Buddhist logicians identified but could not formalize in their own terms.

What non adds: the detection functions

In discrete ternary logic ({T, U, F} = {+1, 0, −1}), `non` acts as a **detector for uncertainty**:

- non(T) = non(+1) = −1 = F
- non(U) = non(0) = +1 = T
- non(F) = non(−1) = −1 = F

That is: non(x) = T if and only if x = U. This is the detection function **J__U**.

From J_U and the existing operators, we can construct the other two detectors:

- **J__T(x)** = and(x, not(non(x))): detects whether x = T.
  - x=T: and(T, not(F)) = and(T, T) = T (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(F, not(F)) = and(F, T) = F (OK)
- **J__F(x)** = and(not(x), not(non(x))): detects whether x = F.
  - x=T: and(F, not(F)) = and(F, T) = F (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(T, not(F)) = and(T, T) = T (OK)

With J_T, J_U, and J_F available, we can build **case expressions**: "if x = T then A, if x = U then B, if x = F then C" for any constants $A, C \in \{T, F\}$:

```
or(and(J_T(x), A), or(and(J_U(x), B), and(J_F(x), C)))
```

The completeness result

Theorem. {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.

Proof sketch. Given any f whose output is always definite (T or F), express it as a disjunction over all input tuples where f returns T. Each such tuple can be detected using J_T, J_U, J_F on individual variables, combined with `and`. The disjunction of these detections yields f. QED.

The one thing {and, or, not, non} *cannot* produce is U as output from definite inputs – because all four operators, given inputs in {T, F}, produce outputs in {T, F}. This is arguably correct: U represents genuine uncertainty, and should not be manufactured from certain data.

A Two-Axis Epistemic Algebra

The four operators constitute a two-axis epistemic algebra:

- **Axis 1: belief polarity** – handled by `not`. Flips the sign; preserves magnitude. Operates on the direction of commitment.
- **Axis 2: commitment strength** – handled by `non`. Erases the sign; inverts magnitude. Operates on how strongly anything is held, regardless of direction.

`and` and `or` combine values along both axes simultaneously (min and max over signed certainty). `not` and `non` decompose the axes: one rotates direction, the other measures grip.

The formal results above establish three things:

1. `non` is not derivable from the classical ternary operators {and, or, not}. It breaks the regularity barrier.
2. Adding `non` yields bivalent functional completeness: {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.

3. `non` enables *detection* of uncertainty rather than mere propagation. Without it, Kleene logic can pass uncertainty through but can never see it.

This is a genuine structural enrichment of Strong Kleene logic.

See also

- Philosophy – pramana theory, tetralemma, and the Buddhist motivation for non-affirming negation.

Training

This document covers the full training infrastructure for WikiOracle's NanoChat integration: how raw data is preprocessed into structured training examples, how DegreeOfTruth governs learning, the dynamic systems interpretation, and the online training pipeline that ties it all together.

DegreeOfTruth (DoT)

DegreeOfTruth is a bipolar scalar on -1 .. $+1$ that measures how well a user's TruthSet agrees with the server's collected truth.

```
DegreeOfTruth = 2 × mean(agreement_i) - 1    for shared entries
```

where:

```
agreement_i = 1 - |server_trust_i - client_trust_i| / 2
```

The bipolar range encodes both true and false statements:

- **DoT = +1**: the user's claims fully agree with the server – the exchange is true. Train at full learning rate.
- **DoT = −1**: the user's claims fully contradict the server – the exchange is false. Train at full learning rate (learning what is *not* true is as valuable as learning what *is* true).
- **DoT ≈ 0**: no shared entries, or perfect cancellation – nothing to learn. Skip training.

Both poles (+1 and −1) train at full strength via |DoT|; only the zero crossing results in a skip. The sign encodes direction (agree/disagree), not magnitude.

This is a placeholder. A future version should incorporate `compute_derived_truth` (operator propagation) before comparison.

Dynamic Systems Perspective

With a bipolar DegreeOfTruth (-1 .. $+1$), the training loop forms a **dynamic equation with both poles and zeros**: DoT = +1 and DoT = −1 are attracting poles where the system learns at full strength (truths and refuted falsehoods respectively), while DoT = 0 is a zero – an equilibrium point where no learning occurs.

This structure resembles a **Hopfield network**, where the energy landscape has stable attractors (memorised patterns) and unstable saddle points. In our system:

- The **TruthSet** plays the role of the weight matrix, encoding the collective memory of what is true and what is false.
- Each **training step** is a state transition that pushes the model toward one of the attractors (truth or refutation).
- The **zero crossing** (DoT ≈ 0) is the energy barrier between attractors – the point of maximum uncertainty where the system has insufficient signal to commit to either direction.

As the server TruthSet accumulates entries from multiple users, the poles and zeros of this dynamic equation shift. The slow-moving average merge ensures that attractors are stable under small perturbations

(anti-capture), while strong consensus can still move the landscape over time. This is analogous to the annealing process in Hopfield networks, where the energy landscape gradually settles into deeper minima as more patterns are stored.

Zero-Mean Balance

The DoT-annealed training algorithm has a **zero-mean balance** property: over many interactions with diverse DoT values, the expected net gradient contribution tends to zero. This arises because:

- DoT = +1 and DoT = −1 both train at full strength but push in opposite directions (learning truth vs. learning falsehood).
- DoT near 0 contributes nearly nothing (the sigmoid zero-crossing).
- Over a diverse population of users, the average DoT tends toward zero if the TruthSet is balanced.

This is directly analogous to the **symmetric energy wells** of Hopfield networks. In a Hopfield network, stored patterns and their complements are both stable attractors. In our system, truths and refuted falsehoods are both stable attractors, and the zero-crossing is the energy barrier between them.

The EMA weight anchoring adds a third stabilizing force: regardless of the DoT distribution, the model is always gently pulled back toward its checkpoint state. This is analogous to the **temperature decay** in simulated annealing – as training progresses, the system becomes increasingly resistant to perturbation while allowing deeper learning within established wells.

Sigmoid Warmup as Annealing Schedule

The sigmoid warmup schedule serves as a **cooling schedule** for the annealing process. When online training is first enabled:

1. The TruthSet is empty or sparse – DoT values are unreliable.
2. The warmup suppresses the learning rate to near-zero.
3. As interactions accumulate, the TruthSet gains signal.
4. The warmup ramps up, allowing the model to learn from now-reliable DoT values.
5. At steady state (step » warmup_steps), the warmup factor is ~1.0 and training proceeds at full configured strength.

This mirrors the annealing schedule in physical systems: high temperature (high exploration, low commitment) → low temperature (low exploration, high commitment to learned patterns).

Sensation – Preprocessing Pipeline

`bin/sensation.py` transforms plain-text conversations into XML-tagged training data so that NanoChat learns WikiOracle's structured protocol. The name follows the epistemological pipeline: **Sensation → Perception → Cognition** – raw input data (sensation) is structured and tagged before it reaches the model (cognition).

Sensation works for both batch retagging of the NanoChat SFT corpus (`identity_conversations.jsonl`, ~14K conversations) and dynamic training examples from the online training pipeline.

JSONL Record Types

The output JSONL uses four record types, splitting the client identity into separate User and Server records:

Type	Tag	Purpose
`user`	`<User>`	User identity – username, uid, timestamp
`server`	`<Server>`	Server identity – name, version, timestamp
`conversation`	`<Conversation>`	Messages with `<Q>` (query) and `<R>` (response) pairs

Type	Tag	Purpose
`truth`	`<Truth>`	Extracted factual claims with trust and spacetime

Inside message content, user messages are wrapped in `<Q>...</Q>` and assistant messages in `<R>...</R>`. Each sentence within a message is independently classified as `<fact>` or `<feeling>`:

```
<R><feeling>That is a great question!</feeling>
<fact trust="0.5"><place>Paris</place>Paris is the capital of France.</fact>
<feeling>I hope that helps.</feeling></R>
```

Korzybski IS Detection

Alfred Korzybski (*Science and Sanity*, 1933) observed that the English copula "is" conflates several distinct relations:

- **IS of identity**: "Socrates is a man"
- **IS of predication**: "The sky is blue"
- **IS of existence**: "There are eight planets"

Each asserts something verifiable about the world – a *fact* bound to a specific spacetime context. "The cup is on the table" is only true at a particular place and time; at a different spacetime it may not be.

The heuristic classifier in `sensation.py` detects these patterns:

Subtype	Pattern	Example
Identity	"X is/are [a/an/the] Y"	"Socrates is a man"
Predication	"X is/are ADJ"	"The sky is blue"
Existence	"there is/are X"	"There are 8 planets"
Mereological	"X contains/includes Y"	"Water contains hydrogen"
Quantity	"X has/have N Y"	"I have 3 cats"
Definition	"X is called/known as Y"	"A polygon is defined as…"

Sentences without an IS pattern, or with subjective markers ("I think", "maybe", "might be"), questions, or meta-discourse ("That's a great question") default to `<feeling>`. Auto-detected facts receive `trust=0.5` – a conservative default that flags them for future verification. Spatiotemporal binding is expressed via `<place>` and `<time>` child elements inside `<fact>` or `<feeling>` tags.

Usage

Batch-convert a corpus:

```
make preprocess
# or: python bin/sensation.py corpus input.jsonl output.jsonl
```

Classify a single sentence:

```
python bin/sensation.py tag "Paris is the capital of France."
# → Classification: fact (identity)
# → Tagged: <Q><fact trust="0.5">Paris is the capital of France.</fact></Q>
```

In the online training pipeline, `response.py` calls `preprocess_training_example()` automatically before each `/train` POST, so all training examples are XML-tagged without manual intervention.

Online Training

Purpose

Enable continuous online training of NanoChat where every interaction can trigger a weight update. The learning rate is governed by DegreeOfTruth (see above) which evaluates how well the user's claims fit the collective evidence.

WikiOracle accomplishes this by:

1. Maintaining a **server-owned TruthSet** (`truth.xml`) that accumulates facts from all users.
2. Computing a **DegreeOfTruth** (-1 .. $+1$) per interaction.
3. Using |DegreeOfTruth| to modulate the **learning rate** of a one-step online training pass in NanoChat.

Conversations are **not** stored on the server. Only **knowledge** truth entries are retained. The server TruthSet contains knowledge facts, operators, authorities, and references – no feelings, provider entries, or **news facts** (spatiotemporally bound observations).

News facts (those with `<place>` or `<time>` child elements carrying real values) are session-only. Persisting them would risk "worldline capture" – an observer could reconstruct a user's physical trajectory through spatiotemporal observations. The `filter_knowledge_only()` function in `bin/truth.py` enforces this boundary.

User Identity

Each user is identified by a pseudonymous GUID derived deterministically from `client_name` in the state file (UUID-5 in the WikiOracle namespace). The stable value is stored as `header/client_id` in XML (the runtime dictionary exposes it as `client_id`) and is used when merging truth entries into the server corpus.

Pipeline

The pipeline is staged so the user gets a response first; truth merging and training happen after the response is delivered.

Stage 1 – Respond

1. Receive the user's query and TruthSet.
2. Use truth entries for RAG as usual.
3. Return the response to the user.

Stage 2 – Compute DegreeOfTruth

4. Score the user's TruthSet against the server's current truth (formula above).

Stage 3 – Update server TruthSet

5. Filter client truth per the Entanglement Policy (Freedom.md):
   - When `store_concrete` is false (default), only universal facts pass through (`filter_knowledge_only()`).
   - Entries with identifiable content are always filtered regardless of `store_concrete` (`detect_identifiability()`).
   - Feelings never reach the merge (`_is_server_storable()` rejects them).
   - Operators may compose facts or feelings as operands.
   - Feelings are also stripped from training messages (`strip_feelings_from_training()` in `bin/sensation.py`).
6. Merge the surviving truth entries into the server TruthSet (`truth.xml`):
   - **Match found**: nudge the server entry's trust toward the incoming value using a slow-moving average: `server_trust += merge_rate × (client_trust - server_trust)`
   - **No match**: insert the entry with the stated trust value.
   - Entries are restricted to facts, operators, authorities, and references. Feelings and provider entries are not stored.

Stage 3a – TruthSet Trimming

When the server TruthSet exceeds `truth_max_entries` (default 1000), the merge step trims the table by removing entries with `|trust|` closest to 0.0 (no information value), keeping entries with highest `|trust|` (strongest signal, positive or negative). This is checked during the merge step, not as a separate operation. The trim count is logged.

The `truth_max_entries` parameter is configured under `server.training` and defaults to 1000. It remains available through the XML config/editor; the current Settings dialog does not expose it as a separate control.

Stage 4 – Tag and Train

7. Preprocess the training example through Sensation (`sensation.py`) to add `<Q>`/`<R>` and `<fact>`/`<feeling>` XML tags.
8. Strip `<feeling>` blocks from training messages (`strip_feelings_from_training()` in `bin/sensation.py`) – feelings must never train model parameters (Entanglement policy, Freedom.md).
9. Call NanoChat's online training endpoint (`POST /train`) with the tagged prompt, DegreeOfTruth, and training hyperparameters.

Device Configuration

Online training runs on the device specified by `server.training.device` in the config file (`config.xml`). Valid values:

- `cpu` (default) – safe for the WikiOracle production server
- `cuda` – use NVIDIA GPU if available
- `mps` – Apple Metal Performance Shaders (macOS)
- `auto` – probe CUDA → MPS → CPU and use the best available

The model is moved to the training device for the gradient step, then moved back to the inference device afterward.

Training Algorithm – DoT-Annealed Selective Training

The `/train` endpoint (`bin/nanochat_ext.py`) implements a carefully designed online training algorithm that prevents weight collapse while enabling meaningful learning from each interaction.

Consistent AdamW Optimizer All parameter groups use **AdamW** consistently. The batch training regime uses Muon (Newton-Schulz orthogonalization) for transformer matrices, but online training uses AdamW for all groups because:

1. **No persistent optimizer state** – the optimizer is created fresh each `/train` call. Muon's advantage comes from accumulated momentum; without persistent state, its Newton-Schulz orthogonalization provides limited benefit.
2. **Effectively sign-SGD** – AdamW without momentum history is effectively sign-SGD: simpler, faster, well-understood.
3. **Independent interactions** – each `/train` call is fully independent. A bad gradient cannot poison future steps because there is no momentum history to contaminate. The effective step size is bounded by `lr`.

Parameter Groups The optimizer creates six parameter groups that mirror the production training regime in `nanochat/nanochat/gpt.py:GPT.setup_optimizer()`:

Group	Base LR	Parameters
`lm_head`	0.0027	Output projection weights (language model head)
`wte`	0.136	Token embedding weights

Group	Base LR	Parameters
`value_embeds`	0.136	Value residual stream embeddings
`resid_lambdas`	0.005	Per-layer residual scaling scalars
`x0_lambdas`	0.5	Skip-connection blending scalars
`transformer_h`	0.02	All transformer block matrix parameters (attention, MLP, norms)

Parameters not matching any named group are included in `transformer_h`. All groups use the same AdamW optimizer – no Muon.

Learning Rate Modulation The effective learning rate for each parameter group is computed as:

```
lr_effective = lr_base × lr_scale
```

where:

```
lr_scale = (truth_weight × |DoT| + (1 − truth_weight)) × sigmoid_warmup(step)
```

The **truth_weight** parameter (0.0-1.0) controls how much DoT gates the learning rate:

`truth_weight`	`lr_effective`	Behavior
0.0	`lr_base × warmup`	Vanilla SFT – full LR regardless of DoT. No truth bias.
0.5	`lr_base × (0.5 × abs(DoT) + 0.5) × warmup`	Half-gated – DoT attenuates but never fully suppresses.
1.0	`lr_base × abs(DoT) × warmup`	Fully DoT-gated – zero DoT means zero learning.

`truth_weight` is configured in `server.truthset` and is not a separate control in the current Settings dialog. At `truth_weight=0`, truth sources are omitted from provider grounding and DoT does not gate training. At `truth_weight=1`, grounding is fully enabled and DoT governs learning.

Sigmoid Warmup The sigmoid warmup schedule prevents early random updates from corrupting the model when online training is first enabled:

```
sigmoid_warmup(step) = 1 / (1 + exp(−k × (step − midpoint)))
```

Step	Warmup value	Effect
0	~0.007	Nearly zero – first interactions barely train
midpoint	0.5	Half strength
2 × midpoint	~0.993	Nearly full strength – training ramp is complete

The `warmup_steps` parameter (default 50) is the sigmoid midpoint. The steepness parameter `k` is fixed at 0.1.

This schedule ensures that the first few interactions after enabling online training have minimal weight impact, allowing the TruthSet to accumulate signal before the model commits to learning from it.

Gradient Clipping After the backward pass, gradients are clipped to prevent catastrophic single-step weight changes:

```
torch.nn.utils.clip_grad_norm_(all_params, max_norm=grad_clip)
```

The `grad_clip` parameter (default 1.0) bounds the global gradient norm. The actual gradient norm after clipping is returned in the `/train` response as `grad_norm` for monitoring.

If `grad_norm` consistently hits the clip threshold, this indicates either: (a) the learning rate is too high for the current data, or (b) the training example is an outlier. Both are handled gracefully by clipping – the step still proceeds, just with bounded magnitude.

EMA Weight Anchoring (Anti-Capture) After each optimizer step, model weights are blended back toward the **checkpoint anchor** using exponential moving average:

```
for p, anchor in zip(model.parameters(), anchor_params):
    p.data.lerp_(anchor, anchor_effective)
```

where:

```
anchor_effective = anchor_decay × truth_weight
```

The anchor weights are the original checkpoint weights, initialized on the first `/train` call. The `anchor_decay` parameter (default 0.001) controls the blend-back rate.

Key properties:

- **truth_weight=0**: `anchor_effective = 0` – no anchor pull. Weights drift freely (vanilla SFT mode).
- **truth_weight=1**: `anchor_effective = anchor_decay` – full anchor pull. Weights are always tugged back toward the checkpoint.
- The anchor prevents **gradual drift** while allowing meaningful learning. Over many steps, the weights can move significantly from the checkpoint, but each individual step is bounded.
- **Reset via `make nano_restart`** – reloads both the model and the anchor, resetting the training state.

The anchor is stored in `app.state.anchor_params` as a list of cloned parameter tensors. The step count is stored in `app.state.train_step_count`.

Training Flow (Per `/train` Call)

```
POST /train
  |- Clamp DoT to [-1, +1], truth_weight to [0, 1]
  |- If truth_weight > 0 and |DoT| < 1e-6 -> skip (no signal)
  |- Acquire model worker from pool
  |- Tokenize messages (bos + user_start/end + assistant_start/end)
  |- If < 2 tokens -> skip (empty)
  |- Initialize EMA anchor weights (first call only)
  |- Move model to training device
  |- Build 6 parameter groups (fresh each call)
  |- Compute lr_scale = (tw x |DoT| + (1 - tw)) x sigmoid_warmup(step)
  |- Scale each group's LR: lr_base x lr_scale
  |- Create fresh AdamW optimizer
  |- Forward pass -> cross-entropy loss
  |- Backward pass
  |- Gradient clipping -> grad_norm
  |- Optimizer step
  |- EMA anchor blend-back (if truth_weight > 0)
  |- Increment step count
  |- Move model back to inference device
  `- Return {status, loss/gain, step, grad_norm}
```

The response uses key `"loss"` for positive DoT (learning truth) and `"gain"` for negative DoT (learning falsehood). Both are the cross-entropy loss value; the key name indicates the semantic direction.

Configuration Summary

Parameter	Config path	Default	Description
`truth_weight`	`server.truthset.truth_weight`	0.7	DoT gating strength (0=vanilla SFT, 1=full DoT)
`warmup_steps`	`server.training.warmup_steps`	50	Sigmoid warmup midpoint
`grad_clip`	`server.training.grad_clip`	1.0	Max gradient norm
`anchor_decay`	`server.training.anchor_decay`	0.001	EMA blend-back rate
`truth_max_entries`	`server.training.truth_max_entries`	1000	Max server TruthSet size
`device`	`server.training.device`	`"cpu"`	Training device

See Config.md Section Server for the full configuration reference.

Truth-Modulated Loss (MentalModel) In the basicModel's `MentalModel`, the training loss is further modulated by **luminosity** and **universality** to prevent learning irrational or unkind propositions:

```
totalLoss = totalLoss * (1 + LuminosityWeight * (1 - luminosity)
                         + UniversalityWeight * (1 - universality))
```

- **Luminosity** measures truth set coherence (see Logic.md Section Luminosity). Low luminosity increases loss.
- **Universality** measures the Golden Rule via SVO/OVS swap (see Ethics.md Section Universality). Low universality increases loss.

Both weights default to 0.1, making the effect gentle. This operates independently of the DoT-annealed training described above – DoT gates the learning rate, while luminosity/universality modulate the loss magnitude.

See Config.md Section MentalModel Configuration for the XML parameters.

SFT Corpus Preparation

The `prepare_sft_corpus()` function in `bin/sensation.py` provides batch conversion of the NanoChat SFT corpus to XML-tagged format with feelings stripped. This is the batch equivalent of `preprocess_training_example()` and is used when retraining NanoChat from scratch with WikiOracle's protocol:

```
make build_sft
# or: python bin/sensation.py sft input.jsonl output.jsonl
```

Each input line is a JSON array of `{"role": ..., "content": ...}` message dicts. Output lines are the same shape but with content XML-tagged using `<Q>`/`<R>` and `<fact>`/`<feeling>` tags, with feelings stripped (matching the online training pipeline).

Server TruthSet

The server TruthSet is stored as `truth.xml` in the same XML format used for state files (WikiOracle State). Each stored entry is a typed truth element:

```
<fact id="..." title="..." DoT="0.8" time="..." place="Athens">
  All men are mortal.
</fact>
```

Resolution: Before entries reach the server TruthSet, they are resolved by `resolve_entries()` in `bin/truth.py`:

- `<reference>` → `<fact src="domain">text</fact>` (domain preserved for deeper lookup)
- `<authority>` → list of `<fact src="domain">content</fact>` (fetched from remote, trust scaled)
- `<provider>` → `<feeling>` (provider responses are treated as feelings until providers can report truth claims with DoT)
- `<fact>`, `<feeling>`, `<logic>` → pass through unchanged

Entry types stored after resolution: `<fact>` (knowledge – no `<place>`/`<time>` children) and `<logic>` entries (operators wrapped in `<logic><and|or|not|non>...</logic>`).

Entry types **not** stored: `<feeling>`, `<provider>`, unresolved `<reference>`, unresolved `<authority>`, and (when `store_concrete` is false) news facts with `<place>` or `<time>` child elements. Content matching identifiability patterns (PII) is always excluded.

News facts are spatiotemporally bound – persisting them risks worldline capture. Whether to store them is a user choice controlled by `store_concrete` in config.xml (default false), consistent with Zero-Knowledge / Selective Disclosure principles. See `filter_knowledge_only()` in `bin/truth.py` and `detect_identifiability()` for PII detection. See Freedom.md for the full policy.

The server TruthSet includes the user GUID as a trust entry, so the server can track per-user trust alongside factual claims.

Anti-Capture

The online training system has multiple layers of defense against weight collapse and capture by any single user:

TruthSet level:

- Entries are merged with a **slow-moving average** (`merge_rate`, default 0.1), so no single user can instantly override collective truth.
- Disproven entries naturally drift toward -1 as contradicting evidence accumulates from other users.
- **TruthSet trimming** (`truth_max_entries`, default 1000) removes low-signal entries ($|\text{trust}|$ near 0), keeping the TruthSet focused on strong signal.

Training level:

- **DegreeOfTruth gates the learning rate** – claims that diverge from consensus (DoT near 0) have minimal training impact.
- **Gradient clipping** (`grad_clip`, default 1.0) prevents any single training step from making catastrophic weight changes.
- **EMA weight anchoring** (`anchor_decay`, default 0.001) continuously blends weights back toward the checkpoint, preventing gradual drift. The anchor pull is scaled by `truth_weight`, so at full truth-gating the model is always tugged back toward its initialized state.
- **Sigmoid warmup** (`warmup_steps`, default 50) prevents the first few interactions from corrupting weights before the TruthSet has accumulated sufficient signal.
- **Fresh optimizer per call** – no persistent momentum state means a bad gradient cannot poison future steps. Each interaction is independent.
- **Consistent AdamW** – without accumulated Muon state, AdamW is effectively sign-SGD with bounded step size.

Operational level:

Manual rollback is available via Makefile targets:

- `make checkpoint-pull` – rsync SFT checkpoints from the remote WikiOracle server to `output/checkpoints/` for safekeeping.
- `make checkpoint-push` – restore checkpoints from backup, then `make wo-restart` to reload weights.

The intended workflow: pull a checkpoint before enabling online training, then push to restore if capture is detected.

Dissonance Detection and Pluralistic Truth (TODO)

Detecting and resolving dissonance within the server TruthSet is left for future work. The goal is to support a higher-dimensional truth-space where contradictory claims can coexist when they originate from different perspectives.

For example: "the world was created in seven days" and "the world was created over millions of years" could both be maintained as true from their respective perspectives. This requires embedding perspective alongside truth value so that the TruthSet becomes a manifold rather than a flat list.

In the current consensus model, a DoT ≈ 0 simply means "nothing to learn" and the training step is skipped. In a future **pluralistic** model – where the same claim can be true in context c_1 and false in context c_2 – a DoT of 0 may instead indicate that **user feedback is needed** to disambiguate which context applies before training should proceed.

Possible approaches include context/perspective tags on entries, truth-space embeddings with frame clustering, conditional truth values indexed by worldview, or explicit user prompts to resolve ambiguity when the TruthSet produces conflicting signals.

OpenClaw Integration

OpenClaw is WikiOracle's multi-channel front-end (Slack, Discord, Telegram, etc.). The TypeScript extension at `openclaw/extensions/wikioracle/` registers WikiOracle as a native OpenClaw provider by spawning `bin/wo` for each query.

The extension provides three capabilities:

1. **Provider** – WikiOracle appears in OpenClaw's provider selector alongside OpenAI, Anthropic, etc. Selecting it routes all messages through the WikiOracle server's full pipeline.
2. **Command** (`/wo <message>`) – Direct CLI-style access from any OpenClaw channel, bypassing the agent/LLM layer.
3. **Tool** (`wikioracle_query`) – Lets OpenClaw agents invoke WikiOracle programmatically during agentic runs.

The message flow through the training pipeline:

1. OpenClaw receives a message from any channel (Slack/Discord/etc.).
2. The wikioracle extension spawns `bin/wo` with the message.
3. `bin/wo` sends `POST /chat` to the WikiOracle server.
4. WikiOracle responds (Stage 1: provider routing, truth RAG).
5. Stages 2-4 execute server-side: DoT computation, truth merge, Sensation preprocessing, and NanoChat online training.
6. `bin/wo` prints the response to stdout; the extension captures it and relays it back to the originating channel.

In stateful mode (default), the WikiOracle server owns session state – conversation context persists across messages without client-side storage. In stateless mode, state is serialized to a local XML file via `bin/wo -f`.

The training pipeline treats OpenClaw messages identically to direct HTTP or web UI clients – the same DoT computation, TruthSet merge, Sensation preprocessing, and `/train` call apply. The `bin/wo` CLI is transparent to the training system.

Extension configuration

```
// In OpenClaw's config file (~/.openclaw/config.json5 or project-level)
{
  plugins: {
    entries: ["wikioracle"],
    wikioracle: {
      woPath: "/path/to/WikiOracle/bin/wo",
      serverUrl: "https://127.0.0.1:8888",
      insecure: true,    // skip TLS verification for local dev
      stateful: true,    // server owns session state
      token: "...",      // optional bearer token
    },
  },
}
```

See `openclaw/extensions/wikioracle/openclaw.plugin.json` for the full config schema.

Server TruthSet Visibility (Debug Mode)

When `DEBUG_MODE` is enabled and online training is active, the `/chat` response includes a `server_truth` key containing the server's TruthSet entries formatted as authority-tagged entries:

```
{
  "server_truth": [
    {
      "type": "authority",
      "id": "entry-id",
      "title": "...",
      "trust": 0.8,
      "content": "<fact>...</fact>",
      "source": "wikioracle"
    }
  ]
}
```

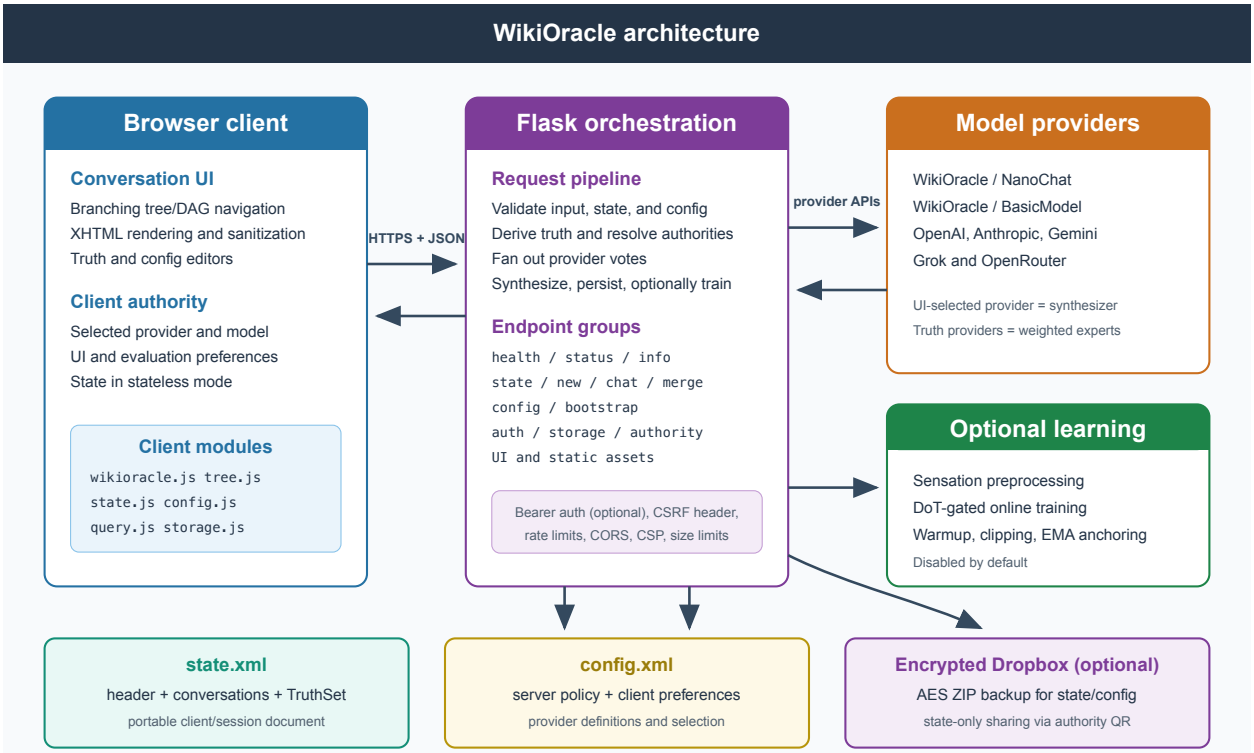
The client displays these entries in the TruthSet with a visual marker (server badge, different border style). Entries are deduplicated by `id` on each debug refresh.

The `server_id` field in `config.xml` under `<server>` provides a stable identifier for this server instance (default: `"wikioracle"`).

Implementation

System Overview

WikiOracle is a Flask orchestration layer between a browser/CLI client and one or more model providers. It combines a recursive conversation graph, typed truth, provider voting, encrypted storage integration, and optional online learning.



Components

Layer	File(s)	Responsibility
Flask application	`bin/wikioracle.py`	Routes, runtime contracts, authentication/CSRF/rate limiting, Dropbox OAuth, encrypted storage, and static UI serving
Configuration	`bin/config.py`	Baseline/override XML loading, provider registry, client-safe projection, environment/runtime settings, XML writer
State	`bin/state.py`	XML/legacy JSON loading, tree/DAG normalization, selection, serialization, atomic writes, collision-safe merge
Conversation graph	`bin/graph.py`	Lookup, ancestry, context paths, selection, flattening, and traversal helpers
Response pipeline	`bin/response.py`	Prompt bundles, truth/provider coordination, adapter calls, HME fan-out, conversation construction, online-training dispatch
Truth engine	`bin/truth.py`	Typed truth normalization, Strong Kleene `and/or/not/non`, authority/reference resolution, DoT, privacy classification, server-truth merge
Sensation	`bin/sensation.py`	Fact/feeling tagging, Korzybski IS classification, spatiotemporal labeling, corpus/SFT preparation
NanoChat extension	`bin/nanochat_ext.py`	`/train` route, DoT-annealed AdamW, warmup, gradient clipping, and EMA anchoring
BasicModel service	`basicmodel/bin/serve.py`	Local OpenAI-compatible BasicModel inference on port 8001 by default

Layer	File(s)	Responsibility
OpenClaw bridge	`bin/wo`, `openclaw/extensions/wikioracle/`	CLI and provider/command/tool integration for OpenClaw channels
Browser shell	`client/index.html`, `client/*.css`	Settings, tree/chat panels, dialogs, editor surfaces, responsive layout
Browser controller	`client/wikioracle.js`	State transitions, message rendering, branching, selection, provider requests
Browser state/config	`client/state.js`, `client/config.js`	Browser persistence plus XML config conversion
Browser graph	`client/graph.js`, `client/tree.js`	Conversation merge/traversal and D3 rendering
Browser utilities/storage	`client/util.js`, `client/storage.js`, `client/query.js`	Editors, settings, Dropbox/QR workflow, and API requests
Schemas	`data/state.xsd`, `data/config.xsd`	Canonical XML contracts
Tests	`test/test_*.py`	State/config roundtrip, stateless contract, security, voting, truth, training, and UI-string coverage

HTTP API

Every route is prefixed by the effective `url_prefix`. POST requests require `X-Requested-With: WikiOracle`. When `WIKIORACLE_API_TOKEN` is set, non-public routes also require the configured bearer token.

Core and Status

Method	Path	Purpose
GET	`/health`	Flask liveness check
GET	`/nanochat_status`	Probe the configured local WikiOracle/NanoChat upstream
GET	`/basicmodel_status`	Probe BasicModel on its configured/default endpoint
GET	`/server_info`	Report stateless mode, prefix, and training availability
GET	`/bootstrap`	Return seed state and client-safe config
GET	`/info`	Return state/schema/provider/startup-merge diagnostics

State, Chat, and Config

Method	Path	Purpose	Stateless behavior
GET	`/state`	Read canonical or in-memory state	Returns memory/seed state
POST	`/state`	Replace state	Replaces memory state; no disk write
POST	`/new`	Reset to an empty session	Replaces memory state
GET	`/state_size`	Report local state-file size	Reports seed-file size if present

Method	Path	Purpose	Stateless behavior
POST/OPTIONS	`/chat`	Process one conversation turn	Requires `state` and `runtime_config`; returns full state
POST/OPTIONS	`/merge`	Merge state payloads or adjacent import files	Rejected because writes are disabled
GET	`/config`	Return the client-safe canonical config	Allowed
POST	`/config`	Replace only the `client` section and persist	Rejected because the shared server config cannot be mutated

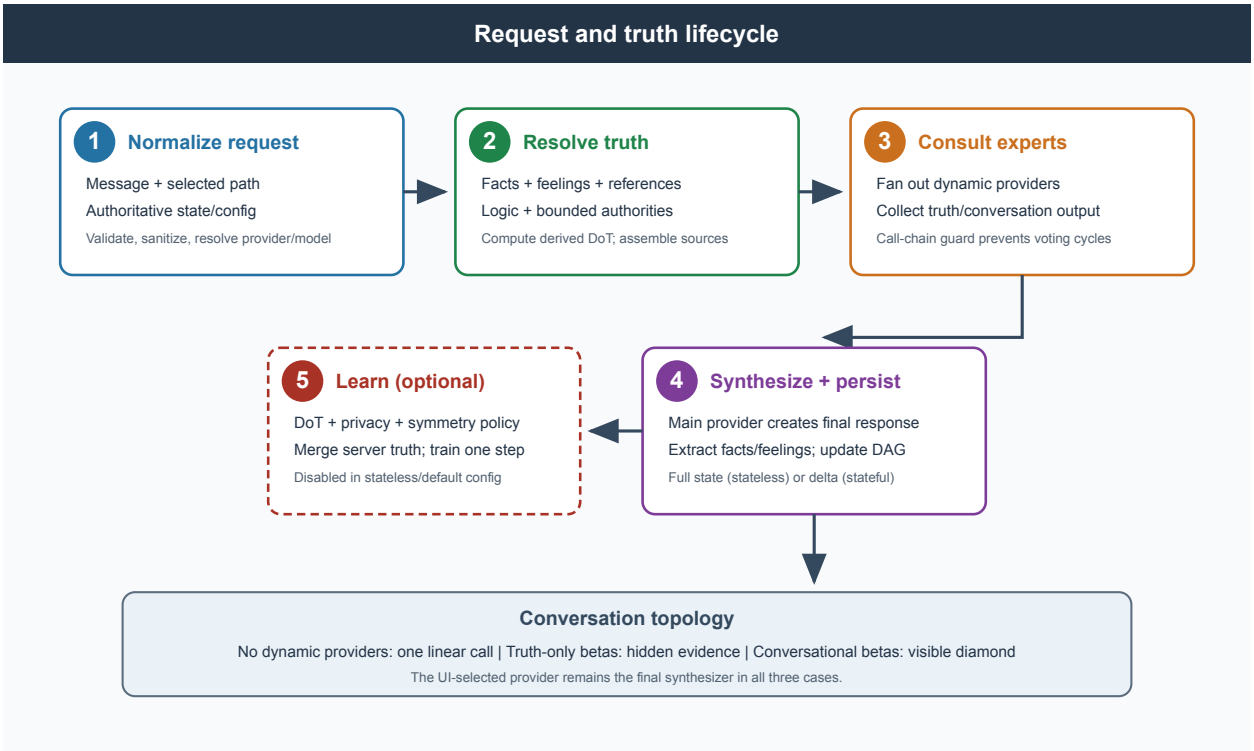
Dropbox and Authority Storage

Method	Path	Purpose
GET	`/auth/dropbox/start`	Begin Dropbox OAuth2
GET	`/auth/dropbox/callback`	Complete OAuth2 and save tokens in the Flask session
GET	`/auth/dropbox/status`	Report whether Dropbox is configured/connected
POST	`/auth/dropbox/logout`	Clear Dropbox tokens from the session
GET	`/storage/status`	Check for encrypted config/state bundles
POST	`/storage/save`	Encrypt and upload state/config; optionally return authority QR data for state-only sharing
POST	`/storage/load`	Download, decrypt, and return/persist state or client-safe config
POST	`/authority/conversations`	Fetch conversations from an authority URL, optionally with a decryption key

UI Assets

Method	Path	Purpose
GET	`/`	Serve `client/index.html` without caching
GET	`/<path>`	Serve an existing asset under `client/` when its extension is allowlisted

Request and State Contracts



Contract	Client sends	Server reads	Server returns	Durable authority
Stateful `/chat`	Message, routing IDs, query config, and optionally client truth	State/config from disk, with supplied truth overriding state truth for the turn	Display text plus selected conversation delta	Local `state.xml`; client retains its own truth copy
Stateless `/chat`	Message, routing IDs, complete authoritative state, and runtime config	Request only	Display text plus complete updated state	Browser/CLI client

The stateless contract never uses disk or a previous in-memory state as an implicit input to `/chat`. The stateful contract persists the complete updated state, but its chat response deliberately omits truth and config.

Chat Routing

Request fields	Routing behavior
`conversation_id`	Continue an existing conversation
`branch_from`	Create a child branch below the specified conversation
Neither	Create a root conversation
Empty message at root/terminal/pending branch	Create a valid continuation/branch turn without a user message

When conversational beta providers participate, the graph becomes a diamond: a query node fans out to beta child conversations, and one final conversation is shared as a child of all participating beta nodes.

Truth and Provider Pipeline

The pipeline does not call a `dynamic_truth()` helper. Instead, it normalizes direct sources, computes logic, resolves provider entries, and assembles one final `ProviderBundle`.

Step	Key implementation	Effect
Direct-source extraction	`static_truth()`, `direct_truth_sources()`	Extract facts, feelings, and references from the client TruthSet
Logic derivation	`compute_derived_truth()`	Recompute `and`, `or`, `not`, and `non` entries and attach derived certainty
Authority/reference resolution	`bin/truth.py` resolvers	Fetch only allowlisted sources with size/depth limits
Beta consultation	`evaluate_providers()`	Query dynamic truth providers; separate conversational and truth-only contributions
Main synthesis	`build_query()`, `_call_provider()`	Send history, truth, derived evidence, and beta output to the UI-selected provider
Structured extraction	`_extract_direct_truths()`	Separate `<conversation>` display output from returned facts/feelings
Graph update	`process_chat()`	Add a linear turn or construct a voting diamond
Optional learning	`process_chat()` plus `nanochat_ext.py`	Merge permitted truth and dispatch a bounded training step

Structural entries are not treated as prose claims. Logic yields derived DoT, authorities yield imported evidence, and providers yield expert contributions. With no dynamic provider entries, the pipeline makes one main-provider call.

Thought-Free Mode

Target	Behavior
External LLM adapters	Adds the non-discursive constraint prompt to the system bundle
BasicModel	Sends `thought_free=true`; the BasicModel service applies its restricted grammar path for the request
Browser display	Strips supported thinking blocks before display according to client preference

The preference originates at `config.client.thought_free`, enters the query bundle, and reaches `_call_provider()`.

Security Middleware

Control	Implementation
Request size	Flask `MAX_CONTENT_LENGTH` from `WIKIORACLE_MAX_STATE_BYTES`
Message length	`WIKIORACLE_MAX_INPUT_LEN`
Input guard	`guard_input()` rejects configured prompt-injection patterns
Authentication	Optional exact bearer-token match
CSRF	Required `X-Requested-With: WikiOracle` on POST
Rate limit	Sliding per-IP limit; chat and default RPM are configurable
Browser policy	Restrictive CSP and allowlisted CORS origins
Static files	Extension allowlist plus resolved-path containment check
State files	Optional symlink rejection and atomic replace

State Library Reference

Function	Purpose
`state_to_xml(state)`	Serialize normalized state to typed XML
`xml_to_state(text)`	Parse typed XML into normalized runtime state
`atomic_write_xml(path, state)`	Temporary-file, `fsync`, atomic-rename write
`load_state_file(path)`	Read XML or legacy monolithic JSON
`find_conversation(conversations, id)`	Recursive tree/DAG lookup
`get_ancestor_chain(conversations, id)`	Resolve all ancestors for context/navigation
`get_context_messages(conversations, id)`	Build ordered path context
`add_message_to_conversation(...)`	Append a message
`add_child_conversation(...)`	Add a branch
`remove_conversation(...)`	Delete a subtree
`merge_llm_states(base, incoming)`	Merge state with deterministic collision-safe identifiers
`ensure_minimal_state(state)`	Normalize required defaults and selection invariants

Config

WikiOracle configuration is XML loaded by `bin/config.py` and described by `data/config.xsd`. `data/config.xml` is the shipped baseline; an optional project-root `config.xml` is deep-merged over it. Scalars and lists in the override replace baseline values, while nested dictionaries merge recursively.

Canonical Structure

Section	Authority	Contents
`<server>`	Operator/server	Runtime mode, truth policy, evaluation/training defaults, URL allowlist, Dropbox app credentials, provider definitions, and shared prompts

Section	Authority	Contents
`<client>`	Browser/client	UI preferences, per-session evaluation choices, provider/model selection, storage preference, and per-provider API keys

```
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="config.xsd">
  <server>...</server>
  <client>...</client>
</config>
```

The browser receives a client-safe projection. Server-only Dropbox attributes and any provider key found in `server.providers` are removed; configured provider definitions are augmented with the model choices used by the UI. Values under `client.providers`, including client-owned API keys, remain browser-visible.

Server

Identity and Runtime

Field	Type	Shipped value	Description
`server_name`	String	`WikiOracle`	Human-readable server label; optional in the XSD
`server_id`	String	`wikioracle`	Persistent server identity; a missing ID is generated in writable mode
`stateless`	Boolean	`true`	Disable state/config writes and require authoritative state/config on `/chat`
`url_prefix`	String	Empty	Prefix applied to every route for reverse-proxy deployments
`truthset`	Section	See below	Truth admission and grounding policy
`evaluation`	Section	See below	Provider evaluation defaults
`training`	Section	See below	Optional continuous-learning defaults
`allowed_urls`	Section	See below	Outbound provider/authority allowlist
`dropbox`	Element	Empty credentials	Server-only Dropbox OAuth app attributes
`providers`	Section	Six definitions	Shared prompt fields and upstream provider definitions

Runtime precedence for the stateless flag is CLI `--stateless`, then `server.stateless`, then `WIKIORACLE_STATELESS`. The route prefix uses CLI `--url-prefix`, then `WIKIORACLE_URL_PREFIX`.

TruthSet Policy

Field	Type	Default	Description
`truth_symmetry`	Boolean	`true`	Check value claims for asymmetric harm under identity exchange
`store_concrete`	Boolean	`false`	Permit spatiotemporally bound facts in the server TruthSet
`truth_weight`	Decimal 0-1	`0.7`	Controls whether/how strongly TruthSet sources enter the prompt and how strongly DoT gates online learning

truth_weight	RAG behavior	Online-learning behavior
0.0	Truth sources are omitted	DoT does not gate the learning rate; EMA anchor effect is zero
0.7	Default weighted grounding	DoT substantially modulates learning and anchoring
1.0	Full grounding	Learning is fully DoT-gated and uses the configured anchor strength

Evaluation Defaults

Field	Type	Default	Description
temperature	Decimal 0-2	0.7	Sampling temperature
max_tokens	Positive integer	128	Maximum response tokens requested from the main provider
timeout	Positive integer	120	Provider request timeout in seconds
url_fetch	Boolean	false	Allow URL content to be incorporated during evaluation
thought_free	Boolean	false	Optional server default for non-discursive output; the client has its own preference field

The request can override provider, model, temperature, URL fetching, thought-free mode, and selected truth/training controls. Values are clamped or normalized in `process_chat()`.

Training

Training is disabled by default and is also unavailable in stateless mode.

Field	Type	Default	Description
enabled	Boolean	false	Master switch for post-response truth merge and online training
truth_corpus_path	String	data/truth.xml	Server truth corpus path
truth_max_entries	Integer 100-10000	1000	Maximum corpus entries before low-information trimming
alpha_base	Decimal	0.01	Base learning rate
alpha_min	Decimal	0.001	Minimum learning-rate floor
alpha_max	Decimal	0.1	Maximum learning-rate ceiling
merge_rate	Decimal	0.1	Moving-average rate when client truth updates an existing server entry
device	auto, cpu, cuda	cpu	Online-training device
dissonance_enabled	Boolean	true	Enable contradiction/dissonance handling
operators_dynamic_enabled	Boolean	true	Enable dynamic operator processing
warmup_steps	Positive integer	50	Midpoint of the sigmoid training warmup
grad_clip	Decimal > 0	1.0	Maximum gradient norm
anchor_decay	Decimal 0-1	0.001	EMA pull toward checkpoint parameters

When `enabled=false`, the post-response DegreeOfTruth, server-truth merge, and online-training stages are skipped. Message-level fact/feeling parsing and response rendering still operate.

Allowed URLs

`allowed_urls` is a repeated prefix allowlist used for authority lookups and dynamic provider endpoints.

URL class	Rule
HTTPS	Allowed only when the full URL begins with a configured prefix
Loopback HTTP	Allowed only for `127.0.0.1` or `localhost`, and only when prefix-matched
`file://`	Denied unless an explicit `file://` prefix is allowlisted; API-key file resolution is additionally confined to its data allowlist and rejects symlinks/traversal
Other schemes	Denied

Dropbox

```
<dropbox app_key="..." app_secret="..."/>
```

Attribute	Required by XSD	Exposure
`app_key`	Yes	Server only; removed from `/config` and `/bootstrap`
`app_secret`	Yes	Server only; removed from `/config` and `/bootstrap`

Dropbox OAuth tokens are stored in the Flask session cookie context; the encryption password supplied for a save/load operation is not stored in configuration or session state.

Provider Definitions

Provider definitions live under `server.providers`. The element uses an explicit `<name>` child, not a `name` attribute.

Shared Prompt Fields

Field	Type	Purpose
`context`	XHTML	Shared system context
`output`	String	Output-format instructions
`truth_context`	XHTML	Role context for truth-only beta providers
`conversation_context`	XHTML	Role context for conversational beta providers

Provider Fields

Field	Required	Description
`name`	Yes	User-facing registry key used by client selection
`type`	Yes	Adapter type: `wikioracle`, `openai`, `anthropic`, `gemini`, `grok`, or `openrouter`
`username`	No	Account/display metadata
`url`	Yes	Upstream endpoint
`model`	Yes	Server default model

Field	Required	Description
`timeout`	Yes	Per-provider timeout
`streaming`	Yes	Streaming capability flag
`api_key`	No in schema	Legacy server-side position; canonical client keys belong under `client.providers` and the server parser omits this field from definitions

```
<providers>
  <context>Return strictly valid XHTML.</context>
  <output>Use conversation, fact, and feeling elements.</output>
  <provider>
    <name>OpenAI</name>
    <type>openai</type>
    <username>you@example.com</username>
    <url>https://api.openai.com/v1/chat/completions</url>
    <model>gpt-4o</model>
    <timeout>120</timeout>
    <streaming>false</streaming>
  </provider>
</providers>
```

Shipped Providers

Name	Type	Shipped model	Endpoint family
WikiOracle	`wikioracle`	`nanochat`	Local OpenAI-compatible endpoint
OpenAI	`openai`	`gpt-4o`	OpenAI chat completions
Anthropic	`anthropic`	`claude-sonnet-4-6`	Anthropic messages
Gemini	`gemini`	`gemini-2.5-flash`	Google Generative Language
Grok	`grok`	`grok-3-mini`	xAI OpenAI-compatible endpoint
OpenRouter	`openrouter`	`google/gemma-3-4b-it:free`	OpenRouter chat completions

The model selector also receives a code-maintained list of supported choices from `_PROVIDER_MODELS`; the configured `model` remains the fallback for each provider.

Client

Client Fields

Field	Type	Default	Description
`storage`	Element	Empty	Optional `state_key` attribute reserved for client storage selection
`temperature`	Decimal 0-2	`0.7`	Client evaluation preference
`url_fetch`	Boolean	`false`	Client URL-fetch preference
`thought_free`	Boolean	`false`	Client thought-free preference
`ui`	Section	See below	Browser layout and interaction preferences
`providers`	Section	See below	Provider/model selection and API keys

UI Preferences

Field	Type	Default	Description
`layout`	`horizontal`, `vertical`	`horizontal`	Main panel arrangement
`theme`	`system`, `light`, `dark`	`light`	Color theme
`divider_pos`	Integer 0-100	`0`	Saved tree/chat divider position
`swipe_nav_horizontal`	Boolean	`true`	Horizontal swipe navigation
`swipe_nav_vertical`	Boolean	`false`	Vertical swipe navigation
`confirm_actions`	Boolean	`false`	Confirm destructive operations

Client Provider Settings

Field	Type	Purpose
`default_provider`	String	Selected provider name; must match a server provider definition
`default_model`	String	Selected model override
`provider/name`	String	Provider name associated with a key
`provider/api_key`	String	Client-owned key for that provider

```
<client>
  <temperature>0.7</temperature>
  <url_fetch>false</url_fetch>
  <thought_free>false</thought_free>
  <ui>
    <layout>horizontal</layout>
    <theme>light</theme>
    <divider_pos>0</divider_pos>
    <swipe_nav_horizontal>true</swipe_nav_horizontal>
    <swipe_nav_vertical>false</swipe_nav_vertical>
    <confirm_actions>false</confirm_actions>
  </ui>
  <providers>
    <default_provider>WikiOracle</default_provider>
    <default_model>nanochat</default_model>
    <provider>
      <name>WikiOracle</name>
      <api_key>...</api_key>
    </provider>
  </providers>
</client>
```

Runtime Selection Precedence

Setting	Resolution order
Provider	Request `config.provider` -> `client.providers.default_provider`
Model	Request `config.model` -> `client.providers.default_model` -> selected server provider `model`
Main-provider API key	Request/runtime client provider key -> loaded `client.providers.<name>.api_key`

Setting	Resolution order
Dynamic truth-provider key	Embedded provider-entry key (literal or allowlisted `file://`) -> matching configured provider key -> selected provider-family environment fallback where implemented
Temperature	Request `config.temp` -> `server.evaluation.temperature`, clamped to 0-2

The browser stores the normalized config in both `sessionStorage` and `localStorage` for tab-close durability. In stateful mode, `POST /config` accepts only the incoming `client` section; the server section remains authoritative on disk. Stateless servers reject config writes.

Runtime Environment Variables

These variables configure the Flask process and its transport. XML still supplies provider definitions and model defaults.

Variable	Default	Purpose
`WIKIORACLE_STATE_FILE`	`state.xml`	Canonical state path
`WIKIORACLE_BASE_URL`	`http://127.0.0.1:8000`	Local WikiOracle/NanoChat base URL fallback
`WIKIORACLE_API_PATH`	`/chat/completions`	Local upstream path
`WIKIORACLE_BIND_HOST`	`127.0.0.1`	Flask bind interface
`WIKIORACLE_BIND_PORT`	`8888`	Flask bind port
`WIKIORACLE_SSL_CERT`	Host-derived path under `~/.ssl`	TLS certificate
`WIKIORACLE_SSL_KEY`	Host-derived path under `~/.ssl`	TLS private key
`WIKIORACLE_TIMEOUT_S`	`120`	General provider timeout
`WIKIORACLE_MAX_STATE_BYTES`	`20000000`	Flask request/state size cap
`WIKIORACLE_MAX_CONTEXT_CHARS`	`40000`	Merge-context rewrite cap
`WIKIORACLE_MAX_INPUT_LEN`	`50000`	Chat-message character cap
`WIKIORACLE_REJECT_SYMLINKS`	`true`	Reject symlinked state files
`WIKIORACLE_AUTO_MERGE_ON_START`	`true`	Import adjacent `llm_*.xml/.json` files at startup
`WIKIORACLE_AUTO_CONTEXT_REWRITE`	`false`	Append merge deltas to context
`WIKIORACLE_MERGED_SUFFIX`	`.merged`	Suffix for imported files
`WIKIORACLE_ALLOWED_ORIGINS`	Local HTTPS origins	CORS allowlist
`WIKIORACLE_API_TOKEN`	Empty	Optional bearer token
`WIKIORACLE_SESSION_SECRET`	Derived from state path	Flask session signing secret
`WIKIORACLE_RATE_LIMIT_CHAT`	`30`	Chat requests per minute per IP; `0` disables the chat-specific limit
`WIKIORACLE_RATE_LIMIT_DEFAULT`	`120`	Default requests per minute per IP
`WIKIORACLE_STATELESS`	`false` fallback	Stateless fallback when CLI/XML do not decide
`WIKIORACLE_URL_PREFIX`	Empty	Route-prefix fallback

`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, and `GEMINI_API_KEY` are last-resort fallbacks for matching dynamic truth-provider URLs. Main provider calls use the canonical `client.providers` key flow.

OpenClaw Extension

The `openclaw` submodule includes `extensions/wikioracle`, which invokes `bin/wo` and registers a provider, `/wo` command, and `wikioracle_query` tool.

Field	Default	Purpose
`woPath`	`../bin/wo`	WikiOracle CLI path
`serverUrl`	`https://127.0.0.1:8888`	Flask server URL
`insecure`	`true`	Skip verification for the local self-signed certificate
`stateful`	`true`	Use the server-owned state contract
`stateFile`	`state.xml`	Local file used for stateless state or stateful logging
`token`	None	Optional bearer token

CLI Flags

```
python bin/wikioracle.py [--config PATH] [--debug] [--stateless]
                         [--no-ssl] [--url-prefix PREFIX]
                         [serve | merge FILE...]
```

Flag/command	Purpose
`--config PATH`	Load a specified configuration file
`--debug`	Enable verbose diagnostic output
`--stateless`	Disable writes and require client-owned request state/config
`--no-ssl`	Serve plain HTTP instead of local TLS
`--url-prefix PREFIX`	Prefix all application routes
`serve`	Start the Flask shim (default)
`merge FILE...`	Merge portable state files into the canonical state

The authoritative schema is `data/config.xsd`.

State

WikiOracle state is a portable XML document managed by `bin/state.py` and described by `data/state.xsd`. The canonical local filename is `state.xml`; the shipped example is `data/state.xml`.

Grammar

```
State        -> Header + Conversation* + Truth?
Conversation -> title + (Message | Conversation)*
Message      -> content + attachment*
Truth        -> (Fact | Feeling | Reference | Logic | Provider | Authority)*
```

Top-level element	Cardinality	Purpose
`<header>`	Exactly one	Versioning, document identity, client label, and timestamps
`<conversation>`	Zero or more	Recursive conversation roots
`<truth>`	Zero or one	Typed TruthSet entries

```xml
<?xml version="1.0" encoding="UTF-8"?>
<state xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:noNamespaceSchemaLocation="state.xsd">
  <header>...</header>
  <conversation id="c_root">...</conversation>
  <truth>...</truth>
</state>
```

UI preferences and provider selection are configuration, not canonical XSD state fields. They live under `<config><client>`; see Config. For migration compatibility, `bin/state.py` still normalizes and round-trips a legacy runtime `ui` dictionary under the state header. That compatibility extension is not declared by `data/state.xsd` and should not be used in newly authored schema-conformant state files.

Header

Field	Type	Required	Description
`version`	Positive integer	Yes	State grammar version (currently `2`)
`schema`	String	Yes	Schema URL or accepted `state.xsd` reference
`time_creation`	String	Yes	Creation timestamp
`time_lastModified`	String	No	Last serialized timestamp
`title`	String	Yes	Portable state title
`client_name`	String	No	Display name attached to user messages
`client_id`	String	No	Stable client identifier

```xml
<header>
  <version>2</version>
  <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
  <time_creation>2026-07-15T10:00:00Z</time_creation>
  <time_lastModified>2026-07-15T10:05:00Z</time_lastModified>
  <title>Research session</title>
  <client_name>Alice</client_name>
  <client_id>alice-uuid-123</client_id>
</header>
```

Conversations

Each conversation contains one `<title>` followed by any number of direct `<message>` and nested `<conversation>` children. The on-disk order preserves the order in which messages and branches appear.

Conversation attributes

Attribute	Required	Description
`id`	Yes	Unique conversation identifier
`parentId`	No	Parent ID or comma-separated parent IDs for a diamond/merge node
`selected`	No	Boolean selection marker; selected conversations form one root-to-node path

Message structure

Field/attribute	Required	Values or type	Description
`id`	Yes	String	Unique message identifier
`role`	Yes	`user`, `assistant`, `system`	Message role
`username`	Yes	String	Display label for the author/provider
`time`	Yes	String	Message timestamp
`selected`	No	Boolean	Unique selected-message marker across the state
`<content>`	Yes	XHTML	Renderable message content
`<attachment>`	No	Repeated element	Inline data or a URL, with optional `name` and `type`

```
<conversation id="c_root" selected="true">
  <title>Animals</title>
  <message id="m1" role="user" username="Alice" time="2026-07-15T10:00:01Z">
    <content><p>Are dogs mammals?</p></content>
  </message>
  <message id="m2" role="assistant" username="WikiOracle" time="2026-07-15T10:00:02Z">
    <content><p>Yes. Dogs are mammals.</p></content>
    <attachment name="taxonomy.svg" type="image/svg+xml" url="https://example.org/taxonomy.svg"/>
  </message>
  <conversation id="c_breeds" parentId="c_root" selected="true">
    <title>Dog breeds</title>
  </conversation>
</conversation>
```

Tree and DAG behavior

Operation	Runtime helper	Result
Find a node	`find_conversation()`	Recursive lookup by ID
Resolve context	`get_ancestor_chain()` / `get_context_messages()`	Ordered messages from root through the active node
Continue	`add_message_to_conversation()`	Append a message to an existing node
Branch	`add_child_conversation()`	Insert a child conversation
Delete	`remove_conversation()`	Remove a subtree
Vote/merge	Shared child plus multiple `parentId` values	Diamond-shaped DAG that remains navigable from each beta branch

The serializer deduplicates a shared DAG node by ID. The `parentId` list preserves the logical multiple-parent relationship even though XML nesting has one emitted location.

TruthSet

The optional `<truth>` element contains typed entries. Common metadata appears as attributes on each truth element.

Common attributes

Attribute	Applies to	Required	Description
`id`	Every truth kind	Yes	Unique entry ID
`title`	Every truth kind	No	Human-readable label
`time`	Every truth kind	No	Timestamp
`place`	Every truth kind	No	Location metadata
`DoT`	Every kind except feeling	Yes	Degree of Truth on [-1, +1]

Truth kinds

Element	Content model	Purpose
`<fact>`	Mixed XHTML/text	Verifiable claim or observation
`<feeling>`	Mixed XHTML/text	Subjective expression outside the truth lattice
`<reference>`	Exactly one `<a href="...">`	External citation
`<logic>`	One `and`, `or`, `not`, or `non` child	Strong Kleene/fuzzy derivation over `<ref>` or inline operands
`<provider>`	Provider-specific child fields	Dynamic expert, optionally producing a visible conversation branch
`<authority>`	URL plus optional refresh interval	Remote TruthSet/state source

```
<truth>
  <fact id="t_dogs" title="Dogs are mammals" DoT="1.0">
    Dogs are mammals.
  </fact>
  <feeling id="t_preference" title="Preference">
    I prefer cats.
  </feeling>
  <reference id="t_reference" title="Dog reference" DoT="0.8">
    <a href="https://en.wikipedia.org/wiki/Dog">Wikipedia: Dog</a>
  </reference>
  <logic id="t_both" title="Combined support" DoT="0.8">
    <and><ref id="t_dogs"/><ref id="t_reference"/></and>
  </logic>
  <provider id="p_claude" title="Claude" DoT="0.7">
    <api_url>https://api.anthropic.com/v1/messages</api_url>
    <model>claude-sonnet-4-6</model>
    <conversation>true</conversation>
    <timeout>120</timeout>
  </provider>
  <authority id="a_remote" title="Remote knowledge" DoT="0.5">
    <url>https://example.org/state.xml</url>
    <refresh>3600</refresh>
  </authority>
</truth>
```

Operator arity

Operator	Operands	DoT result
`and`	Two or more	Minimum operand DoT
`or`	Two or more	Maximum operand DoT
`not`	Exactly one	Negated DoT
`non`	Exactly one	`1 - 2 * abs(DoT)`

See Logic for the interpretation and completeness argument.

Provider entry fields

Field	Required	Meaning
`api_url`	No	Dynamic provider endpoint
`api_key`	No	Literal key or supported indirection used by dynamic-provider resolution
`model`	No	Model override
`system`	No	XHTML system instruction
`authority`	No	Nested URL and optional refresh configuration
`conversation`	No	Whether the contribution appears as a conversation branch
`timeout`	No	Request timeout
`max_tokens`	No	Response-token cap

Truth Evaluation and Persistence

Phase	Entry kinds	Behavior
Direct truth	Facts, feelings, references	Converted to source records for the provider bundle
Derived truth	Logic	Recomputes DoT from the current operand entries
Dynamic truth	Providers, authorities	Consults experts or imports bounded remote evidence
Server truth merge	Resolved facts and logic	Optional moving-average merge into `data/truth.xml`
Excluded from server truth	Feelings, providers, references, authorities	Must be resolved or intentionally omitted before persistence

The server truth corpus accepts either a `<truth>` root or a `<state>` document containing `<truth>`. It is separate from the client's complete state file.

Runtime Representation

`xml_to_state()` normalizes XML into a Python dictionary. Conversations use `messages[]` and `children[]`; truth entries use a stable envelope (`id`, `title`, `trust`, `content`, and optional metadata). Selection attributes are also exposed through helper fields such as `selected_conversation` and `selected_message`.

XML surface	Runtime shape
Recursive `<conversation>` children	Nested `conversations[]` / `children[]` dictionaries

XML surface	Runtime shape
Typed truth element name	XHTML/XML in `entry["content"]`
`DoT` attribute	Numeric `entry["trust"]`
Conversation/message `selected="true"`	Per-node flags plus derived selected IDs

Serialization and Merge

Function	Purpose
`state_to_xml(state)`	Serialize normalized state to typed XML
`xml_to_state(text)`	Parse typed XML into normalized state
`load_state_file(path)`	Load XML or a legacy monolithic JSON state
`atomic_write_xml(path, state)`	Write through a temporary file, `fsync`, and atomic rename
`ensure_minimal_state(state)`	Add required defaults and normalize tree/selection fields
`merge_llm_states(base, incoming)`	Merge portable state with deterministic collision-safe IDs

The authoritative schema is `data/state.xsd`. Legacy JSON imports remain supported for migration, but newly persisted state is XML.

User Interface

Rendering

The display layer covers HTML structure, CSS classes, D3 node shapes, optimistic UI, and state persistence.

Context for the upstream LLM

When a message is sent, the server needs to provide conversational context to the upstream model. The function `get_context_messages(conversations, conv_id)` walks the **ancestor chain** from the active conversation up to the root, collecting all messages in chronological order.

```
Root conversation      ->  messages: [m1, m2, m3]
  |- Child conv        ->  messages: [m4, m5]
     `- Grandchild     ->  messages: [m6, m7]  <- active

Context sent to LLM: [m1, m2, m3, m4, m5, m6, m7]
```

This gives the LLM the full path of dialogue that led to the current point, without noise from sibling branches.

Tree visualisation (D3)

`tree.js` renders the conversation tree as a top-down hierarchy using `d3.tree()`:

```
conversationsToHierarchy(state.conversations, selectedId)
    ↓
D3 hierarchy data  { id, title, messageCount, questionCount, selected, children }
    ↓
renderTree(hierarchyData, callbacks)
    ↓
SVG with nodes (rects/pills/circles) + curved links
```

The root node is a circle labelled `/`. Selected conversations render as larger labelled rectangles; others as compact pills. The selected node is highlighted with the accent colour.

Chat view

`wikioracle.js → renderMessages()` displays the messages of the currently selected conversation:

1. Look up `selectedConvId` in the tree via `findConversation()`
2. Iterate `conv.messages`, rendering each as a `.message` div with role-based styling
3. Attach drag-to-reorder (HTML5 drag/drop) and right-click context menus to each message
4. Re-render the D3 tree in sync

When no conversation is selected (root view), a placeholder prompts the user to type.

Rendering pipeline (end to end)

```
state.xml on disk
    ↓  [xml_to_state]
In-memory state with nested conversation tree
    ↓  [GET /state]
Client receives state payload
    ↓  [renderMessages]
Chat panel shows selected conversation's messages
    ↓  [conversationsToHierarchy]
D3 hierarchy data
    ↓  [renderTree]
SVG tree visualisation
```

Interactions

Tree panel

Gesture	Action
Click	Navigate – select that conversation, show its messages
Double-click	Open context menu (Branch, Delete)
Right-click	Open context menu (same)
Drag node → drop on node	Merge – append source's messages into target, remove source

A 200ms timer disambiguates click from double-click. Context menus are appended to `document.body` with fixed positioning to avoid clipping by the tree container's `overflow: hidden`.

Keyboard navigation

Key	Action
Arrow Up	Navigate to parent conversation
Arrow Down	Navigate to first child conversation
Arrow Right	Next node in inorder traversal
Arrow Left	Previous node in inorder traversal

Keyboard events are ignored when a text input, textarea, or select element has focus, or when the context overlay is active.

Diamond traversal A diamond (DAG merge) occurs when a conversation has multiple parents (e.g. the final synthesis in an HME vote that merges two beta branches):

```
    root
   /    \
beta1  beta2
  \    /
   final      ← parentId: [beta1, beta2]
```

Arrow Right / Arrow Left perform standard tree preorder traversal. In a DAG the same node appears under multiple parents; it is visited **once per parent** so that each parent's subtree is a complete reading path:

```
→ root → beta1 → final
→ beta2 → final → END
```

Chat panel

Gesture	Action
Right-click message	Context menu (Move up, Move down, Delete)
Drag message	Reorder within the conversation

Merge semantics

Dragging conversation A onto B:

1. Appends A's messages after B's existing messages (preserving order)
2. Re-parents A's children under B
3. Removes A from the tree

Branching

Double-click or right-click a tree node → "Branch" creates a new empty child conversation. The next message typed seeds it. The LLM receives the full ancestor context.

Settings Dialog

The Settings panel (gear icon in the sidebar) configures client identity, provider/model selection, provider trust, layout, theme, and evaluation preferences. The normalized config is persisted under the `wikioracle_config` key in both `sessionStorage` and `localStorage`. Conversation state uses the separate `wikioracle_state` key.

Settings Controls

Control (DOM id)	Value / default	Storage and behavior
Username (`setUsername`)	Text; `User`	`state.client_name`; display name attached to user messages
Provider (`setProvider`)	Select; shipped `WikiOracle`	`config.client.providers.default_provider`; active main provider
Model (`setModel`)	Select; selected provider's configured model	`config.client.providers.default_model`; explicit model override
Provider trust (`setProviderTrust`)	Range 0-1; `0` when absent	Browser copy of `config.server.providers.<name>.trust`; runtime trust metadata for provider-produced truth
Layout (`setLayout`)	Select; `horizontal`	`config.client.ui.layout`; horizontal or vertical panels

Control (DOM id)	Value / default	Storage and behavior
Theme (`setTheme`)	Select; `system` UI fallback	`config.client.ui.theme`; system, light, or dark palette
Temperature (`setTemp`)	Range 0-2; server evaluation default	`config.client.temperature`; sampling temperature
Allow URL fetching (`setUrlFetch`)	Checkbox; `false`	`config.client.url_fetch`; client evaluation preference
Thought-free mode (`setThoughtFree`)	Checkbox; `false`	`config.client.thought_free`; request non-discursive output and the BasicModel thought-free path
Confirm deletes / merges (`setConfirm`)	Checkbox; `false`	`config.client.ui.confirm_actions`; confirm destructive operations

Settings Persistence

Setting family	Stateful mode	Stateless mode
`state.client_name` and conversations/truth	State is updated through `/state` or `/chat`; browser keeps a local copy	Browser state is authoritative and accompanies every `/chat` call
`config.client.*`	Saved locally, then `POST /config` replaces the client section in the writable config override	Saved only in browser storage; `POST /config` is rejected
`config.server.*`	Refreshed from `/config`; client POSTs cannot replace it	Included in the authoritative `runtime_config` returned by bootstrap/local storage
Provider metadata/model lists	Refreshed from `/config` or `/bootstrap`	Stored with the normalized config bundle

Truth weight, concrete-fact storage, maximum truth entries, and training parameters remain available in the XML config/editor but are not separate controls in the current Settings panel.

Server TruthSet Display

In debug mode, when online training is enabled, the client displays server TruthSet entries alongside local truth entries.

Treatment	Value
Left border	3px accent blue
Badge	`SERVER` in small uppercase text
Opacity	0.65

Server truth entries are injected from the `/chat` response's `server_truth` field (debug mode only) and deduplicated by `id` on each refresh.

User Interface Strings

Canonical UI text for the WikiOracle client. A consistency test (`test/test_ui_strings.py`) periodically checks that the strings hard-coded in `client/util.js` match the tables below.

Truth editor

Dropdown labels

Key	Label
feeling	Feeling: subjective, non-verifiable claim
fact	Fact: disprovable assertion about the world
reference	Reference: citation with external link
authority	Authority: pointer to remote TruthSet
provider	Provider: external LLM endpoint

Descriptions

Key	Description
feeling	Feeling -- a subjective, non-verifiable claim (not penalizable).
fact	Fact -- a disprovable assertion with a degree of truth.
reference	Reference -- a citation linking to an external source.
and	AND -- true when all children are true (min trust).
or	OR -- true when any child is true (max trust).
not	NOT -- negation of a child entry.
non	NON -- non-affirming negation (weakens trust toward zero).
provider	Provider -- an LLM API endpoint.
authority	Authority -- a remote knowledge base (XML URL).

Templates

Key	Template
feeling	<feeling>Subjective statement here.</feeling>
fact	<fact trust="0.0">Assertion text here.</fact>
reference	<reference trust="0.0">Link text</reference>
and	<logic><and><ref id=""/><ref id=""/></and></logic>
or	<logic><or><ref id=""/><ref id=""/></or></logic>
not	<logic><not><ref id=""/></not></logic>
non	<logic><non><ref id=""/></non></logic>
provider	<provider trust="0.0"><api_url>https://api.example.com</api_url><model>model_name</model></provider>
authority	<authority trust="0.0"><url>https://example.com/state.zip</url><key>password</key></authority>

Empty state

Key	Text
truth_empty	No truth entries. Add truth here or Open a new state file.

Proposed License

WikiOracle Licensing Architecture

This document proposes a licensing structure for the WikiOracle project designed to preserve software freedom, maintain an open knowledge commons, and enable responsible AI development.

Four-Layer Conceptual Model

WikiOracle separates artifacts into distinct licensing domains.

Layer	Artifact type	Recommended license
Layer 1	Software / engine code	GPL-3.0
Layer 2	Human knowledge content	CC BY-SA 4.0
Layer 3	Structured data	CC0
Layer 3B	Model weights	Apache-2.0

Conceptual summary:

Artifact	Encodes
Code	Algorithms
Data	Facts
Weights	Learned structure
Content	Explanations

This layered separation mirrors successful open knowledge systems while extending them for AI systems.

License Comparison Table

License	Domain	Key Features	Typical Use
GPL-3.0	Software	Strong copyleft, patent protection, source distribution requirement	Infrastructure software
CC BY-SA 4.0	Creative works	Attribution required, derivatives must remain under same license	Wikipedia articles
Apache-2.0	Software / model artifacts	Permissive reuse with patent grant	AI models and developer tools

GPL-3.0

Properties:

- strong copyleft protection
- requires publication of source code
- includes explicit patent protection
- prevents proprietary forks of infrastructure

CC BY-SA 4.0

Properties:

- allows copying and adaptation
- requires attribution
- derivatives must remain under the same license
- widely used for collaborative knowledge projects

Apache-2.0

Properties:

- permissive reuse
- explicit patent protection
- compatible with commercial ecosystems
- widely used for machine learning artifacts

Rationale for Layered Licensing

Software (GPL-3.0)

The WikiOracle engine should remain permanently open. GPL ensures that improvements to the infrastructure remain open-source and cannot be captured by proprietary forks.

Knowledge Content (CC BY-SA)

Human-authored knowledge should remain part of a global commons. CC BY-SA ensures attribution and share-alike behavior so improvements remain public.

Structured Data (CC0)

Structured datasets (truth graphs, semantic relations, dataset exports) benefit from minimal legal friction. CC0 allows unrestricted reuse and avoids attribution chains that complicate machine learning workflows.

Model Weights (Apache-2.0)

Model weights are derived artifacts produced by training. Apache-2.0 allows redistribution, fine-tuning, and commercial use while providing patent protection.

Training Data Provenance Policy

AI systems increasingly require transparent documentation of training sources.

WikiOracle should maintain a provenance record including:

- dataset origin
- license compatibility
- ingestion date
- preprocessing steps
- contributor identity (if applicable)

This allows:

- legal defensibility
- reproducibility
- verification of training inputs

Example provenance record structure:

```
dataset_name
source_url
license
date_ingested
transformation_pipeline
```

Provenance tracking supports both transparency and scientific reproducibility.

AI-Generated Content Policy

AI-generated text occupies a hybrid authorship category.

WikiOracle should treat generated content in three stages:

Stage	License
Raw AI output	CC0
AI draft edited by a human	CC BY-SA
Human-authored content	CC BY-SA

Rationale:

- raw AI output may lack clear authorship
- human editorial involvement establishes attribution
- share-alike preserves the knowledge commons

All AI contributions should record metadata:

```
generation_model
timestamp
human_editor
revision_history
```

This ensures transparency regarding machine involvement.

Summary

WikiOracle licensing stack:

Layer	Domain	License
1	Software	GPL-3.0
2	Knowledge content	CC BY-SA 4.0
3	Structured data	CC0
3B	Model weights	Apache-2.0

The architecture protects the software commons, maintains a global knowledge commons, and supports open AI development.